Wordcount = 19190

UNIVERSITY
OF OSLO

Master's Thesis

NEUROMORPHIC COMPUTING

With Spiking Neural Networks

Brage Wiseth

Departement of Informatics
Faculty of Mathematics and Natural Sciences

Philip Haflinger - Supervisor
Yngve Hafting - Co-Supervisor

2026-04-13

ABSTRACT

The development of modern Deep Learning has achieved unprecedented performance across various domains, yet it remains fundamentally bottlenecked by the energy and memory inefficiencies of the von Neumann architecture. To address these limitations, this thesis investigates Neuromorphic Computing with Spiking Neural Networks (SNNs) as a biologically plausible, highly energy-efficient alternative. By shifting from synchronous, continuous-value matrix multiplications to asynchronous, event-driven sparse computations, neuromorphic systems emulate the physical principles of the biological brain.

This work explores the implementation of these principles on standard CPU/GPU hardware. Two primary methodologies are developed and evaluated on visual classification tasks: (1) an inference-optimized SNN that translates weights from a conventionally trained Fully Connected Network (FCN) using Time-To-First-Spike (TTFS) temporal encoding, and (2) an unsupervised, biologically inspired learning simulator incorporating structural plasticity (dynamic synaptogenesis and pruning). The results demonstrate the viability of temporal coding and local learning rules in extracting meaningful features from visual stimuli, highlighting the potential of neuromorphic algorithms to drastically reduce the computational footprint of artificial intelligence.

ACKNOWLEDGEMENTS & DECLARATIONS

I would like to thank my supervisors and the very kind and helpful comunity at the ROBIN and NANO research groups at the departemnt of informatics.

DECLARATION OF THE USE OF GENERATIVE ARTIFICIAL INTELLIGENCE

In this scientific work, generative Artificial Inteligence (AI) has been used. All data and personal information have been processed in accordance with the University of Oslo's regulations, and I, as the author of the document, take full responsibility for its content, claims, and references. An overview of the use of generative AI is provided below.

The service Gemini, developed by Google, has been used to improve the content of the thesis. Sections of text like a subsection, paragraph or source code for figures was given to the model along with prompts to make the language free of errors and more profesional. The text was then iterated a few times through the model where new prompts were used to get the correct structure and flow of the text. In the case for figures the model was used to speed up the development of the figures with the model generating numbers for positioning elements. The final result was cut out, fact-checked, and partly rewritten by the author.

CONTENTS

GLOSSARY

action potential: Voltage potential in the neuron cell membrane that propagates trough the neurons axon and to its synapses

***AI* – Artificial Inteligence**

***ALU* – Arithmetic Logic Unit**

***ANN* – Artificial Neural Network**

***CNN* – Convolutional Neural Network**

***DAG* – Directional Acyclic Graph**

***DL* – Deep Learning**

***FCN* – Fully Connected Network**

***GLIF* – Generalized Leaky Integrate and Fire**

***GPT* – Generative Pre-trained Transformer**

***LIF* – Leaky Integrate and Fire**

***LLM* – Large Language Model**

***LSTM* – Long Short-Term Memory**

***LTD* – Long-Term Depression**

***LTP* – Long-Term Potentiation**

***MLP* – Multi Layer Perceptron**

***PSC* – Post Synaptic Current**

***RNN* – Recurrent Neural Network**

***SNN* – Spiking Neural Network**

spike: Alias for an action potential that travels trough the network. We refer to a spike as an action potential abtracted into a single event in time

***STDP* – Spike-Timing-Dependent Plasticity**

***SyOPs* – Synaptic Operations**

***TTFS* – Time-To-First-Spike**

***VLSI* – Very Large Scale Integration**

***WTA* – Winner-Take-All**

1 • INTRODUCTION

The development of intelligent machines is a significant objective in modern science and engineering. While the concept has historical roots in philosophy and early automata, the field has transitioned from speculative theory to practical application. Currently, artificial intelligence is central to technological and economic development. Understanding the mechanisms of intelligence and reproducing them in synthetic systems offers the potential for improved analysis of biological minds and the creation of tools for applications ranging from personalized medicine to automated scientific discovery.

In recent years, great strides has been made towards this goal. Deep Learning, which utilizes multilayered neural networks, has exceeded previous performance benchmarks. These systems have demonstrated high proficiency in tasks previously limited to human capability. For example, AlphaFold has addressed complex problems in protein folding [1], reinforcement learning agents have mastered the complexity of games such as Go [1], and Large Language Models have shown capabilities in text generation that approach human fluency. Consequently, AI is increasingly viewed as a general-purpose technology that may influence societal infrastructure.

However, despite these advances, there are significant limitations to the current approach. The success of modern deep learning relies heavily on scaling, which involves increasing data volume and computational power. This strategy is approaching physical and economic boundaries. Training state-of-the-art models consumes substantial energy and results in a large carbon footprint [1]. Although specialized hardware allows for more efficient computations, the underlying architecture and algorithms imposes an intrinsic limit on scalability independent of the underlying hardware. Furthermore, the requirement for massive datasets presents challenges in sourcing and curation. Additionally, evidence suggests that this scaling approach yields diminishing returns. Models often function as statistical correlation engines; they lack common-sense reasoning, struggle with out-of-distribution generalization, and are prone to brittle failure modes [1].

These limitations are evident when comparing artificial systems to biological intelligence. The human brain demonstrates that high-level intelligence is possible without massive energy consumption or dataset sizes. The brain operates on approximately 20 watts [1]. With this limited energy budget, it manages biological functions, processes real-time multi-sensory data, and performs abstract reasoning. In contrast, deep learning models require GPU clusters with significantly higher power requirements to match a fraction of these capabilities. There is also a discrepancy in learning efficiency. Deep learning models are sample-inefficient, often requiring vast numbers of examples to learn a representation. Biological systems, however, are capable of “one-shot” or “few-shot” learning and can acquire new information

without catastrophic forgetting. This suggests the inefficiency of current AI is a paradigmatic issue rather than just an engineering problem.

The proposed direction for addressing these issues involves biological inspiration in both hardware and algorithm design, specifically Neuromorphic Computing. This field attempts to engineer computer architectures that mimic the biological structure of the nervous system. Unlike traditional AI, which runs as software on general-purpose hardware, neuromorphic engineering aims to align the algorithm with the physical substrate. It moves away from clock-driven processing toward asynchronous, event-driven systems. In this paradigm, information is encoded as sparse, discrete events or "spikes". Similar to biological neurons, a neuromorphic processor consumes minimal energy when inactive, processing information only when triggered. This approach is being pursued by both industrial groups, such as Intel's Loihi [1], and academic projects like SpiNNaker [1]. These systems represent a shift from calculation-based machines to those capable of real-time adaptation.

Although neuromorphic systems achieve optimal performance on co-designed platforms—where the algorithm is embedded directly into the hardware—there is significant value in executing neuromorphic algorithms on traditional von Neumann architectures. In this thesis, we explore biologically inspired algorithms deployed on traditional CPU and GPU hardware. We examine how event-driven, biologically plausible computation can address limitations in scalability, data efficiency, and energy consumption, even when simulated on standard processors. We present approaches for efficient information coding and learning algorithms inspired by neural mechanisms. Concretely, this thesis aims to:

- **Investigate Sparse Information Flow**: Explore how information can be encoded and processed using sparse, asynchronous events (spikes) within a neural network.
- **Develop Biologically Inspired Learning**: Explore and evaluate learning algorithms that are suitable for such networks, adhering to the constraints of locality and efficiency.

The succeeding chapter lays the historical and theoretical foundation, covering early neuroscience and the development of artificial neural networks based on simple models of the brain. Following this, we review relevant modern neuroscience literature, extracting key concepts that will inform the methodology. We also provide consise overview on machine learning concepts and frameworks. Finally, we detail the implementation of these principles in a neuromorphic context and evaluate their performance against standard benchmarks.

2 • HISTORY & DEVELOPMENTS

Historically, the understanding of neural tissue was dominated by the reticular theory, which claimed that the brain consisted of a continuous, fused network of nerve fibers. This paradigm was fundamentally challenged by the work of Santiago Ramón y Cajal. Through the application of novel staining techniques, Cajal established the neuron doctrine, demonstrating that the nervous system is composed of discrete, individual cells [1]. Building on these findings, Heinrich Wilhelm Gottfried Von Waldeyer-Hartz proposed the "Neuron Doctrine" and coined the term "neurons" to describe these dicrete cells. Subsequent analysis using electron microscopy has provided irrefutable validation of this discrete cellular structure.

The conceptualization of the brain as a collection of discrete units facilitated the development of mathematical models describing neural function. In 1943, Warren McCulloch and Walter Pitts published A Logical Calculus of the Ideas Immanent in Nervous Activity, introducing the first formal model of the neuron.

The McCulloch-Pitts (M-P) neuron abstracted biological complexity into a binary decision device governed by the following logic:

- **Inputs**: The neuron receives multiple binary inputs, weighted as either excitatory or inhibitory.
- **Summation**: The unit calculates the linear sum of these weighted inputs.
- **Thresholding**: If the aggregate sum exceeds a fixed threshold, the neuron outputs a 1 (firing); otherwise, it outputs a 0 (silence).

McCulloch and Pitts demonstrated that networks of these units could theoretically compute any logical operation (AND, OR, NOT) [1]. This abstraction established the foundational link between biological processes and digital logic, suggesting that neural function could be replicated in electronic hardware. Consequently, the M-P neuron serves as the common ancestor for both computational neuroscience and artificial intelligence.

However, despite its theoretical utility, the original M-P model presented significant functional limitations. The connectivity was static, requiring circuits to be manually designed rather than learned. Furthermore, the restriction to binary weights precluded the modeling of graded signal intensity, preventing the system from capturing the nuance of real-world sensory input.

In 1949, Donald Hebb addressed the critical issue of plasticity in his work The Organization of Behavior. He proposed a theoretical mechanism for synaptic modification, now known as Hebbian learning, which provided a biological basis for how neural networks could adapt over time. Hebb postulated:

> "Let us assume that the persistence or repetition of a reverberatory activity (or "trace") tends to induce lasting cellular changes that add to its stability. ... When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased" [1].

This principle is colloquially summarized as "neurons that fire together, wire together" [1]. Crucially, this describes a local and decentralized learning rule; a synapse does not require a global error signal or external supervision to adjust. It requires only the correlation between the pre-synaptic input and the post-synaptic output. The convergence of the M-P architectural model and the Hebbian plasticity framework established the prerequisite conditions for the development of modern neural networks.

2.1 • THE PERCEPTRON

In 1957, Frank Rosenblatt advanced these theoretical concepts by engineering the Perceptron. The "Mark I Perceptron" was a hardware implementation of the neural model, distinguished by a crucial innovation: a weight-adjustment mechanism based on Hebbian principles. Rosenblatt introduced the perceptron learning rule, an iterative algorithm capable of minimizing error automatically. The system processed an input pattern (e.g., a pixelated character) and produced a binary classification. When the output deviated from the target, the algorithm adjusted the weights proportional to the error: strengthening connections that should have contributed to a correct firing and weakening those that led to false positives.

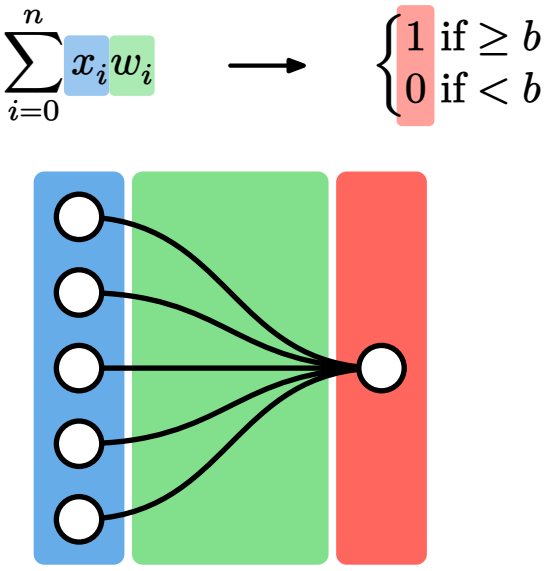

Figure 1: The perceptron model. Inputs x_i are multiplied by weights w_i and summed. If the linear combination $\sum x_i w_i$ exceeds the bias b, the neuron activates.

Consequently, the Perceptron was capable of converging on a solution for any problem where the data was linearly separable. This success generated significant enthusiasm, with contemporary reports suggesting that such machines would soon mimic human consciousness [1].

These expectations were abruptly tempered by theoretical limitations. In 1969, Marvin Minsky and Seymour Papert published Perceptrons, a rigorous mathematical analysis of the architecture. They demonstrated that a single-layer perceptron is fundamentally a linear classifier. While capable of learning operations like AND or OR, it is mathematically incapable of solving the XOR (Exclusive OR) problem. In the XOR case, the classes cannot be separated by a single hyperplane. This proof highlighted a severe boundary on the utility of single-layer networks for complex, non-linear tasks.

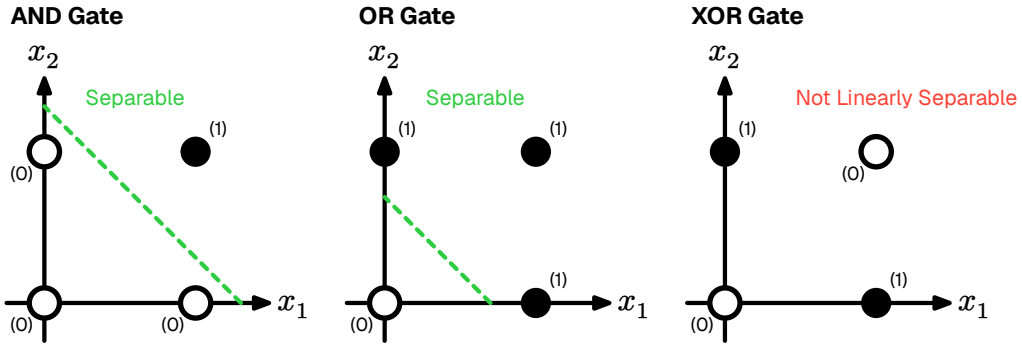

Figure 2: The XOR problem. Unlike AND/OR, the data points for XOR cannot be separated by a single linear boundary.

The publication of Perceptrons coincided with a significant reduction in neural network research funding, a period retrospectively termed the “First AI Winter”. It is worth noting that Minsky and Papert acknowledged that a Multi Layer Perceptron (MLP), a network stacking multiple layers of neurons, could theoretically solve the XOR problem by creating complex, non-linear decision boundaries.

However, a critical algorithmic gap remained: the “credit assignment problem”. While researchers knew that hidden layers could represent complex features, there was no known method to propagate error signals back through the layers to adjust the weights of hidden neurons effectively. Rosenblatt’s rule was mathematically valid only for the output layer. The field remained stagnant until a method for training multi-layer networks could be formalized.

2.2 • DEEP LEARNING

The critique presented by Minsky and Papert precipitated a contraction in funding; despite this, theoretical inquiry persisted. It was widely hypothesized that the limitations of the single perceptron could be overcome by a MLP. By organizing neurons (single perceptrons) into hierarchical layers, the network could theoretically perform successive non-linear transformations on the input space, enabling

the formation of complex decision boundaries. The primary impediment was not the architecture itself, but the absence of a viable learning algorithm.

In a single-layer perceptron, error attribution is immediate: if the output deviates from the target, the error is directly derived from the weights of the output layer. However, in a multi-layer architecture, quantifying the contribution of a specific neuron within the "hidden" layers to the final output error presents a significant challenge. This is formally known as the Credit Assignment Problem [1], and it remained the central theoretical obstacle for over a decade.

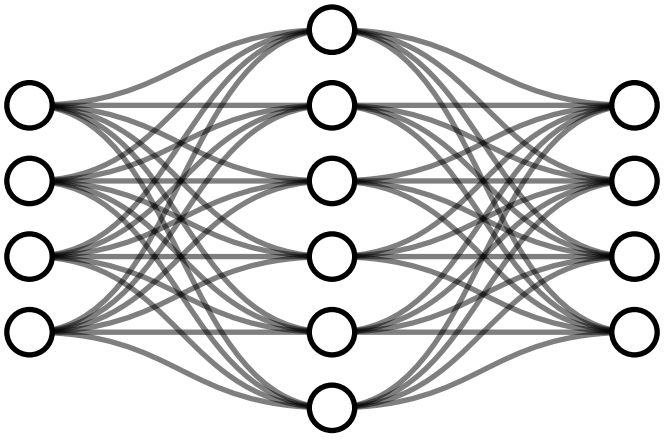

Figure 3: A Multi Layer Perceptron (MLP). By inserting "hidden layers" between input and output, the network can approximate non-linear functions such as XOR. The historical challenge lay in deriving a method to train these intermediate layers.

The solution to this theoretical impasse was popularized in 1986 by Rumelhart, Hinton, and Williams in their seminal paper *Learning representations by back-propagating errors* [1]. They demonstrated that the Chain Rule of calculus could be applied recursively to propagate the error signal from the output layer backwards through the hidden layers. This algorithm, known as Backpropagation, allowed the network to calculate the gradient of the loss function with respect to every weight in the system. Effectively, it provided a mathematical method to tell each hidden neuron exactly how much it contributed to the total error, finally solving the credit assignment problem.

Unlike Hebbian plasticity, which is local and biological, Backpropagation relies on global error signals and precise backward data flow—mechanisms effectively absent in organic tissue. Consequently, the field of Artificial Neural Network (ANN) effectively decoupled from neuroscience. It transitioned into a branch of engineering and applied mathematics, prioritizing statistical optimization over biological realism. Paradoxically, it was this abandonment of biological fidelity that enabled the rapid scaling and performance breakthroughs that followed.

2.2.1 • ACHIEVEMENTS

With the training mechanism solved, the field exploded. The combination of Backpropagation, massive datasets, and GPU hardware led to a "Cambrian Explosion"

of neural architectures, each solving domains previously thought impossible for computers.

The revolution began in earnest with computer vision. Convolutional Neural Networks (CNNs), such as AlexNet (2012) [1] and later ResNet [1], introduced the idea of learning hierarchical features—detecting edges, then shapes, then objects—much like the human visual cortex. This allowed machines to classify images with superhuman accuracy.

Soon after, the focus shifted to sequence data. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) architectures gave machines a short-term memory, enabling breakthroughs in speech recognition and machine translation. However, the true paradigm shift occurred with the introduction of the Transformer architecture in 2017. By utilizing an "attention mechanism" to parallelize the processing of language, Transformers allowed for the training of massive Large Language Models (LLMs) like the Generative Pre-trained Transformer (GPT).

These techniques have even transcended media generation. Deep Learning has solved fundamental scientific problems; notably, DeepMind's AlphaFold utilized these architectures to predict the 3D structure of proteins from their amino acid sequences, a 50-year-old grand challenge in biology [1].

2.2.2 • SHORTCOMINGS

Deep learning's reliance on computational scaling masks fundamental inefficiencies in both its hardware implementation and underlying algorithms. By simulating biological concepts on digital architectures not designed for them, the current paradigm is approaching physical and economic limits.

A primary limitation is the Von Neumann architecture, which physically separates processing units from memory. Deep neural networks, defined by massive matrices of synaptic weights, necessitate constant data transfer. For every inference step, billions of parameters must be fetched from off-chip DRAM, processed, and written back. This creates a severe memory bottleneck where system performance is bounded by bandwidth rather than processing speed [1].

Consequently, the energy cost of moving data significantly exceeds the cost of computation itself. Retrieving a single byte from DRAM consumes approximately three orders of magnitude more energy than performing a floating-point operation [1]. Compounding this hardware friction, the dense matrix multiplications required for training scale quadratically with network size, making the pursuit of trillion-parameter models increasingly unsustainable.

Furthermore, the optimization algorithms driving this scale are fundamentally incompatible with physical biological systems. Backpropagation, while mathematically elegant, relies on a global error signal and suffers from the "weight transport problem"—the requirement that the backward pass utilizes the exact same synaptic

weights as the forward pass. In organic tissue, synapses are unidirectional, and there is no known mechanism for a neuron to access the exact weight of a downstream synapse to calculate a gradient.

While a detailed technical analysis of these inefficiencies is presented in Section 4, the central issue is clear: modern AI prioritizes statistical optimization over physical realism. Overcoming the limitations of the Von Neumann bottleneck and backpropagation requires a paradigm shift toward architectures that inherently co-locate memory and computation.

2.3 • BIRTH OF NEUROMORPHIC

While the artificial intelligence community debated symbolic logic versus connectionism during the "AI Winter," significant developments were occurring in hardware physics. In the late 1980s at Caltech, physicist Carver Mead—a pioneer of Very Large Scale Integration (VLSI) design—began to question the trajectory of digital computing.

Mead observed that while digital computers were becoming exponentially faster, they were also becoming less efficient in terms of energy per operation. He noted that using transistors as rigid, high-power switches to perform boolean logic was energetically wasteful compared to the biological systems they aimed to emulate.

In 1990, Mead published his seminal paper, *Neuromorphic Electronic Systems* [1], coining the term "neuromorphic" to describe hardware that mimics the biological structure of the nervous system. His thesis proposed that rather than simulating neural equations via software on digital computers, engineers should construct physical hardware that exploits the same physical laws as the biological nervous system.

The foundational insight of the field was the physical analogy between silicon physics and ion-channel physics. In standard digital electronics, transistors are operated in "strong inversion," driven by high voltages to act as binary switches. Mead realized that a single transistor, operating in its "subthreshold" region, follows the same exponential Boltzmann statistics that govern the flow of ions through biological channels.

This realization implied that a single transistor could physically compute the nonlinear functions used by biological neurons, but with significantly higher speed and lower power consumption. Consequently, synaptic functions could be implemented by single transistors rather than complex arrangements of logic gates.

To demonstrate this concept, Mead and his doctoral student Misha Mahowald developed the *Silicon Retina* in 1991 [1]. Unlike a standard camera, which captures full frames at fixed intervals (generating redundant data), the Silicon Retina operated asynchronously. It utilized analog circuits to compute spatial and temporal deriva-

tives directly on-chip, outputting discrete “events” only when local light intensity changed.

This event-driven approach solved the redundancy problem inherent in frame-based sampling. If the scene remained static, the system transmitted no data and consumed negligible energy. This demonstrated that by aligning the hardware physics with the computational task, sensory information could be processed with a fraction of the power required by conventional digital systems.

Since the inception of neuromorphic computing, neuroscience has also advanced significantly. While Mead’s early work was based on the physical intuition of the transistor, modern neuromorphic engineering now incorporates a richer understanding of neuronal dynamics, synaptic plasticity, and network architecture. To advance the field, we must combine these foundational hardware insights with the principles of modern mechanistic neuroscience.

3 • BIOLOGICAL PRINCIPLES

The biological brain remains the gold standard for energy-efficient, robust, and adaptive computation. Since the establishment of the Neuron Doctrine, modern neuroscience has uncovered the specific physical mechanisms that underpin this efficiency. To engineer systems that truly rival biological performance, we must transcend the "spherical cow" abstractions of early cybernetics. We cannot simply mimic the brain's output; we must emulate its internal dynamics. This requires viewing the neuron not as a static summing unit, but as it functions in reality: a complex, time-dependent, and event-driven processor.

This section provides a mechanistic overview of the nervous system, translating biological observations into the computational primitives required for neuromorphic engineering. It explores the structural hierarchy of the neuron, the physics of the action potential, and the mathematical models used to capture these dynamics in silicon.

3.1 • NEURON STRUCTURE & FUNCTION

In Section 2 we established the neuron as the fundamental computational unit of the brain. While it shares standard cellular machinery like mitochondria and a nucleus with other cells, it is morphologically specialized for information transmission. A neuron consists of three functional zones:

- **The Input (Dendrites)**: A branching tree structure that collects signals from thousands of upstream neurons. This is where inputs are integrated.
- **The Integration Zone (Soma)**: The cell body where electrical potentials from the dendrites summate.
- **The Output (Axon)**: A long, cable-like structure that transmits the neuron's own signal to downstream targets.

The neuron exhibits a distinct morphological polarization that dictates the direction of information flow. The process begins at the "dendritic arbor", a complex branching structure that maximizes the surface area for synaptic connectivity. These dendrites serve as the primary receptor sites, where neurotransmitters binding to post-synaptic terminals induce local conductance changes. These signals propagate passively toward the soma (cell body), the neuron's central processing unit. The soma acts as an integrator, spatially and temporally summing the incoming synaptic currents. Finally, the processed signal is transmitted via the axon, a singular, elongated projection. In many vertebrate neurons, the axon is insulated by a myelin sheath, which facilitates saltatory conduction—a mechanism that allows high-speed signal propagation over long distances with minimal signal degradation.

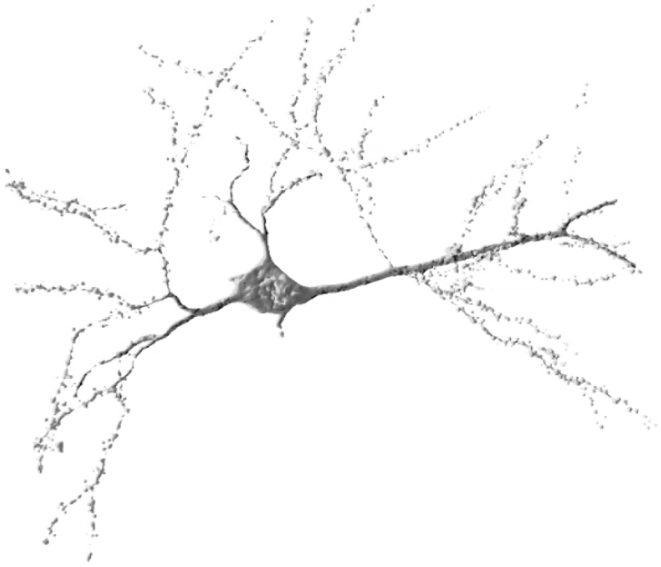

Figure 4: The morphological structure of a biological neuron, illustrating the directional flow of information from dendritic input to axonal output. © Angela Getz, Mathieu Ducros, Daniel Choquet / IINS et BIC / CNRS-Université de Bordeaux-Inserm.

Functionally, the neuron operates as an electrochemical system enclosed by a cell membrane, known as the "lipid bilayer". This membrane is a thin, fatty structure that is impermeable to ions, acting as an electrical insulator. However, the fluids inside and outside the cell are conductive electrolytes. Consequently, the interaction between the insulating membrane and the conductive fluids creates a biological capacitor, capable of storing charge.

By actively pumping sodium (Na^+) out and potassium (K^+) in via the Na^+-K^+ ATPase pump, the cell maintains an electrochemical gradient across this capacitor, resulting in a stable "resting potential" of approximately -70 mV.

Computation occurs through the modulation of this voltage by competing synaptic inputs. Excitatory inputs cause ion channels to open, allowing positive ions to influx; this reduces the negative charge (depolarization) and pushes the potential toward the firing threshold. Conversely, inhibitory inputs activate channels for negative ions (like Chloride, Cl^-), driving the potential away from the threshold (hyperpolarization). The soma integrates these opposing push and pull signals. If the aggregate membrane potential surpasses a critical threshold (approximately -55 mV), the system undergoes a bifurcating phase transition. Voltage-gated sodium channels cascade open, triggering an action potential—a rapid, non-linear depolarization spike that propagates down the axon. This mechanism is governed by the "all-or-nothing" principle: the output is discrete and binary, effectively filtering out sub-threshold noise.

Immediately following a spike, the neuron enters a "refractory period" during which ion gradients are restored, imposing a hard limit on the maximum firing frequency and ensuring the temporal separation of events.

It is important to acknowledge that the biological brain exhibits significant cellular diversity beyond this idealized model. The nervous system contains non-neuronal cells known as “glia”, which provide structural support and manage energy delivery, though they are generally not considered direct participants in fast information transmission. Additionally, while the vast majority of cortical neurons communicate via uniform action potentials (spikes), certain sensory neurons utilize “graded potentials”, where the signal amplitude varies continuously. However, as spiking neurons represent the dominant computational paradigm for information processing in the cortex, this thesis focuses exclusively on the spiking model as the basis for neuromorphic emulation.

3.2 • ACTION POTENTIAL & SPIKE TRAINS

As established in the previous section, the action potential is an “all-or-nothing” event. It serves as the fundamental mechanism by which neurons transmit information. Crucially, the waveform of this event is stereotypical: for a given neuron, every spike exhibits a nearly identical amplitude and duration (typically 1–2 ms), independent of the input intensity that triggered it.

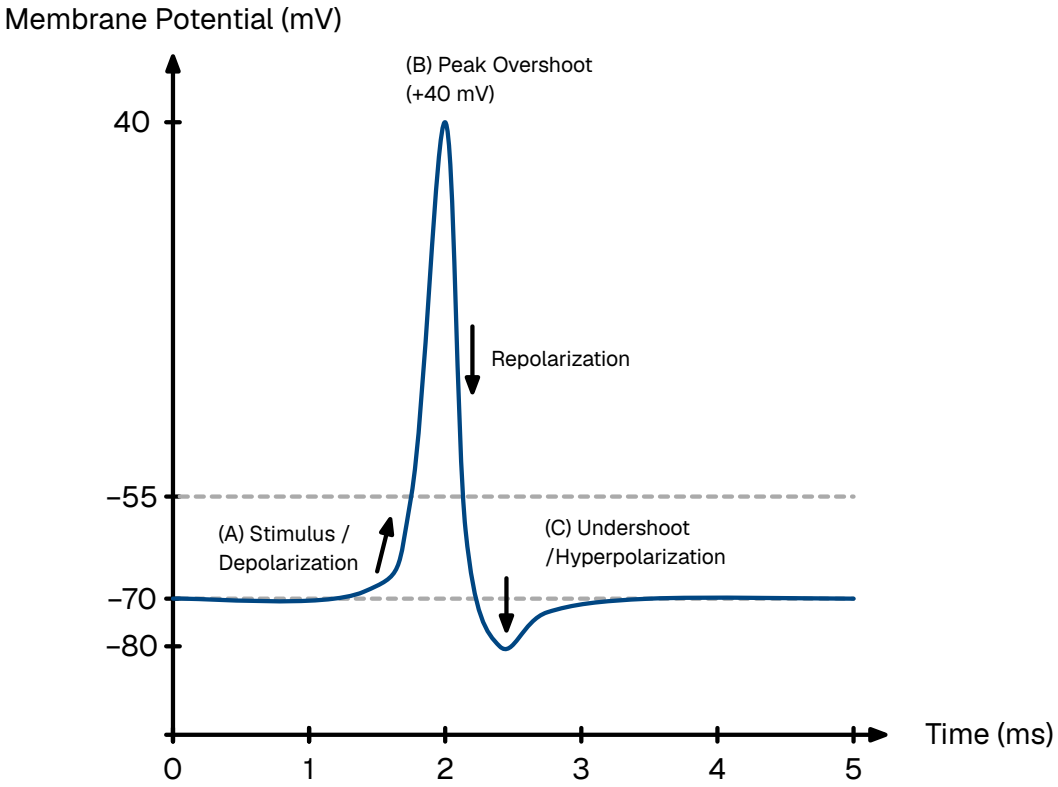

Figure 5: The phases of a typical neuronal action potential. (A) An incoming stimulus depolarizes the membrane past the threshold (−55 mV), triggering a rapid spike. (B) The membrane potential reaches a peak overshoot (+30 mV) before repolarizing. (C) A temporary undershoot (hyperpolarization) occurs before returning to the resting state (−70 mV). The neuron cannot fire during the absolute refractory period (D) and requires a stronger stimulus to fire during the relative refractory period (E).

This biological invariance permits a fundamental simplification in neuromorphic modeling:

> If the spike waveform is invariant across neurons and time, the waveform itself carries no information. Consequently, the information content of the signal is encoded entirely in the precise timing of the spike.

To model this mathematically, we abstract the continuous biophysical voltage trace into a dimensionless point process. We treat the action potential not as a function of voltage over time, but as a singular event occurring at a precise instant, t_f, with negligible duration. The standard tool for this abstraction is the Dirac delta function denoted as, $\delta(t)$.

The Dirac delta is a generalized distribution defined by the property that it is zero everywhere except at the origin, yet integrates to unity. This represents an idealized pulse of infinite height and zero width, containing a finite unit of effect.

$$\delta(t) = \begin{cases} \infty \text{ if } t = 0 \\ 0 \text{ if } t \neq 0 \end{cases}, \quad \int_{-\infty}^{+\infty} \delta(t)\,\mathrm{d}t = 1$$

Equation 1: The defining properties of the Dirac delta function.

Under this formalism, the output of a neuron is modeled not as a continuous signal, but as a "spike train"—a temporal sequence of these Dirac impulses. For a neuron emitting N spikes at times $\left\{t^{(1)}, t^{(2)}, ..., t^{(N)}\right\}$, the output signal $S(t)$ is defined as:

$$S(t) = \sum_{f=1}^{N} \delta\left(t - t^{(f)}\right)$$

Equation 2: A spike train represented as a sum of Dirac delta functions.

This abstraction allows the post-synaptic effect to be modeled using linear systems theory. In neuron models that use this framework, the interaction is treated as instantaneous charge deposition: the arrival of a delta function $\delta\left(t - t_f\right)$ imparts a discrete step-change to the post-synaptic current. This mimics the rapid opening of ion channels without requiring the computational overhead of simulating the complex voltage trajectory. The shift from continuous values to discrete spike trains fundamentally alters the computational paradigm, moving from spatial representations (magnitude-based) to spatio-temporal representations (time-based).

3.3 • NEURON MODELS

In the quest to simulate the brain, there exists a fundamental trade-off between biological realism and computational efficiency. At the high end of the spectrum lie conductance-based models, most notably the Hodgkin-Huxley model. This formalism describes the neuron not as a simple computational unit, but as an electrical

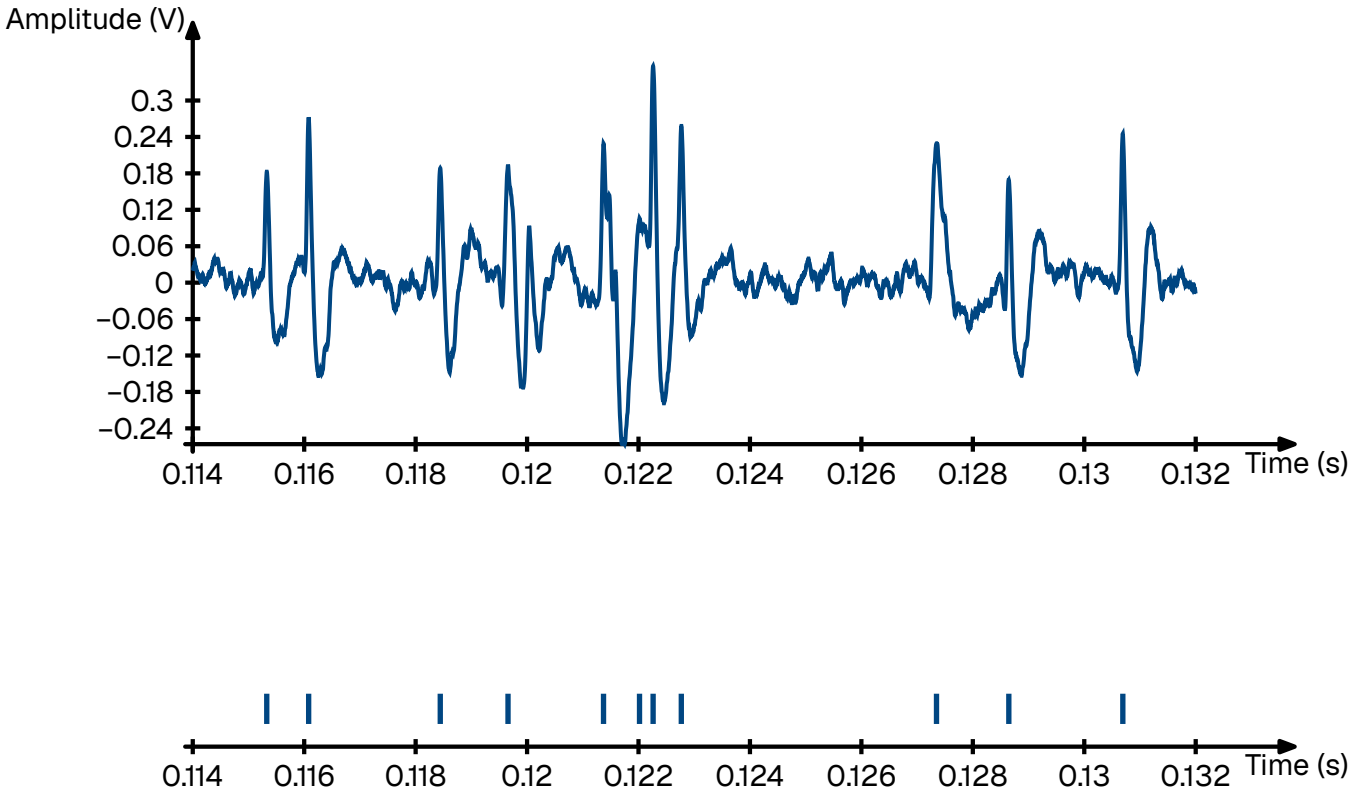

Figure 6: Transformation of continuous membrane voltage (top) into a discrete spike train (bottom).

circuit with variable resistors representing the precise, non-linear opening and closing dynamics of specific ion channels (sodium, potassium, leak) [1].

Large-scale initiatives, such as the Blue Brain Project, utilize even more granular "multi-compartment" models. These simulations treat the neuron as a complex 3D structure, discretizing the dendritic arbor and axon into hundreds of segments to model how current flows through the specific morphology of the cell [1]. While invaluable for pharmacological research, these models are computationally prohibitive for large-scale neuromorphic engineering. Simulating a mere second of biological time for a small network using these equations requires supercomputing resources.

To build practical, scalable neuromorphic hardware, we must abstract these biophysical details into a phenomenological model. We seek a mathematical framework that captures the essential computational properties—integration, leakage, and thresholding—without simulating the underlying molecular physics.

3.3.1 • THE LEAKY INTEGRATE-AND-FIRE (LIF) MODEL

The standard approximation used in neuromorphic engineering is the Leaky Integrate and Fire (LIF) model. This framework aligns perfectly with the "point process" abstraction established in the previous section, as it treats action potentials as instantaneous, discrete events. Its state is defined by a single scalar variable, the membrane potential $u(t)$. The sub-threshold dynamics are governed by a linear

differential equation analogous to a simple RC (Resistor-Capacitor) circuit. We implement this model as the computational baseline for our visual classification tasks in Section 5.3.2 (Model B).

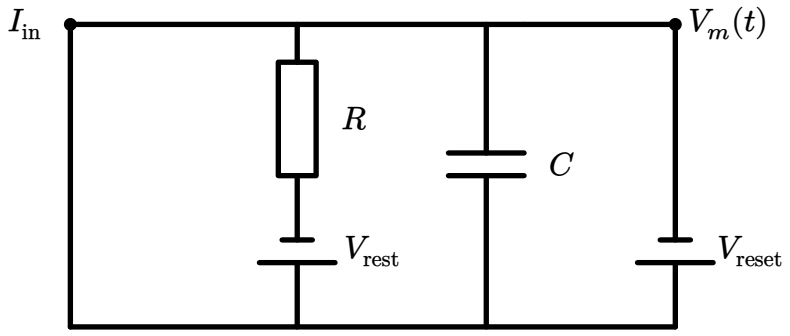

Figure 7:

$$\tau_m \frac{\mathrm{d}u}{\mathrm{d}t} = -(u - u_{\text{rest}}) + RI(t)$$

Equation 3: The Leaky Integrate-and-Fire (LIF) differential equation. The change in voltage is driven by the leak (decay to rest) and the input current.

Where τ_m is the membrane time constant (determining how fast the neuron "forgets"), u_{rest} is the resting potential, R is the membrane resistance, and $I(t)$ is the input current.

Connecting this to the spike train abstraction derived in the previous section, the input current I(t) is not continuous. It is a sequence of discrete events arriving from pre-synaptic neurons j with weight w_j. Mathematically, this is modeled as a sum of Dirac delta functions:

$$I(t) = \sum jw_j \sum f\delta\left(t - t_{j(f)}\right)$$

Equation 4: Synaptic input modeled as a weighted sum of Dirac delta functions.

Because the differential equation is linear below the threshold, we can solve it analytically. The membrane potential $u(t)$ becomes a convolution of the input spike train with the system's impulse response (a decaying exponential kernel). This means the potential at any moment is simply the sum of the decaying traces of all past spikes:

$$u(t) = u_{\text{rest}} + \sum jwj \sum f \exp\left(-\frac{t - t_{j(f)}}{\tau_m}\right)$$

Equation 5: The analytical solution for the membrane potential. The current voltage is the superposition of all past inputs, decayed by time constant τ_m.

The differential equation above describes the continuous sub-threshold dynamics. To complete the model, we must define the discrete "Fire" mechanism. The LIF neuron operates as a hybrid dynamical system: it integrates continuously until a discontinuity is triggered.

When the membrane potential $u(t)$ exceeds a specific threshold voltage ϑ, the neuron emits a spike (a Dirac delta event). Immediately following this event, the

membrane potential is not governed by the differential equation but is forced to a reset value, u_{reset}.

To mimic the biological limit on firing frequency, the model enforces an absolute refractory period, Δ_{ref}. After a spike occurs at time t_f, the neuron is clamped to the resting potential for a duration of Δ_{ref}. During this interval, the differential equation is suspended; the neuron ignores all inputs and cannot fire, regardless of the stimulus intensity.

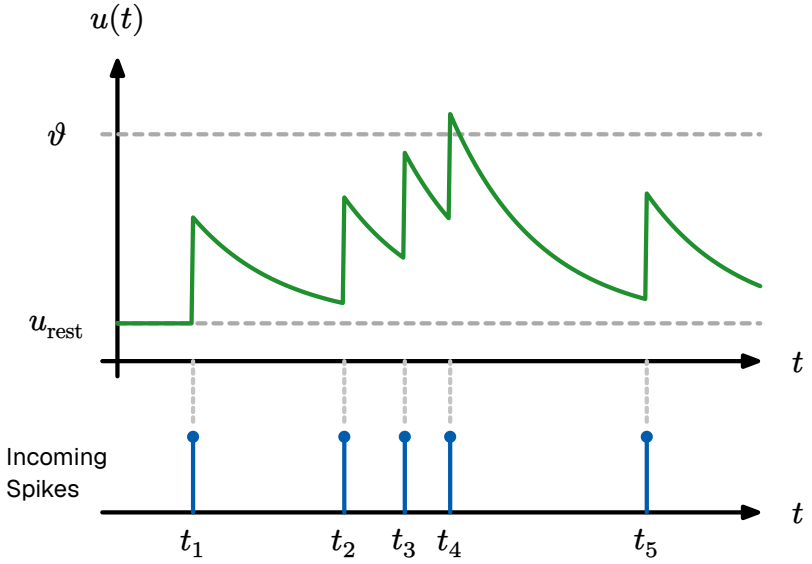

Figure 8: The dynamics of an Leaky Integrate and Fire (LIF) neuron. (A) The membrane potential integrates inputs. (B) Upon crossing the threshold ϑ, a spike is emitted and the voltage is reset. (C) The voltage is clamped during the refractory period.

Mathematically, this firing condition is expressed as:

$$\text{If } \; u(t) > \vartheta \rightarrow \begin{cases} \text{Emit spike:} \;\; S(t) \rightarrow S(t) + \delta(t) \\ \text{Reset voltage:} \;\; u(t) \rightarrow u_{\text{reset}} \\ \text{Pause integration for} \;\; t \in \left(t_f, t_f + \Delta_{\text{ref}}\right] \end{cases}$$

Equation 6: The discrete firing and reset condition.

This equation represents the engine of most neuromorphic algorithms. It defines a system that integrates information over time and leaks it to ensure temporal relevance. However, once $u(t)$ crosses a threshold ϑ, the linearity breaks and the neuron emits a spike and $u(t)$ is manually reset to u_{reset}.

3.3.2 • THE GENERALIZED (ADAPTIVE) LIF MODEL

While the standard LIF model is computationally efficient, its one-dimensional nature limits it primarily to tonic spiking (regular firing under constant input). It struggles to replicate the complex, non-linear behaviors observed in the cortex, such as bursting (clusters of rapid spikes) or spike-frequency adaptation (slowing down after sustained activity).

To capture these dynamics without reverting to the computationally heavy Hodgkin-Huxley equations, we employ the Generalized Leaky Integrate and Fire (GLIF) model. This extends the system by introducing a second state variable, $w(t)$, representing cellular adaptation.

$$\tau_m \frac{\mathrm{d}u}{\mathrm{d}t} = -(u - u_{\text{rest}}) + RI(t) - w$$
$$\tau_w \frac{\mathrm{d}w}{\mathrm{d}t} = a(u - u_{\text{rest}}) - w$$

Equation 7: The Adaptive GLIF system. The adaptation variable w provides negative feedback, enabling complex dynamics like bursting and adaptation.

In this coupled system, w provides a negative feedback loop. Every time the neuron spikes, w increments by a constant b, acting as a physiological drag on the membrane potential. By adjusting the coupling parameters between u and w, this two-dimensional system can be tuned to emulate the full spectrum of biological firing patterns.

It is natural to question whether such a mathematically reduced model can genuinely capture the behavior of biological neurons. While the GLIF model discards the specific ionic mechanisms of the Hodgkin-Huxley equations, empirical validation demonstrates that it retains superior computational dynamics for large-scale modeling.

In the 2008 *Quantitative Single-Neuron Modeling Competition* [1] organized by the INCF, phenomenological models like the Generalized LIF (specifically the Adaptive Exponential Integrate-and-Fire) unexpectedly outperformed highly detailed biophysical models in predicting the precise spike times of real cortical neurons.

This counter-intuitive success is due to parameter sensitivity. Complex conductance-based models have dozens of unobservable parameters that are difficult to tune. In contrast, the GLIF model captures the "net effect" of these mechanisms using macroscopic parameters that can be robustly fitted to data. As demonstrated by Izhikevich (2003), this simple system of two differential equations is capable of reproducing all known firing patterns observed in the mammalian cortex [1]. We leverage this state-dependent adaptation to implement Model D (Section 5.3.2), evaluating its effectiveness in temporal sequence decoding.

Consequently, for the purpose of neuromorphic engineering, the GLIF model represents the optimal trade-off between biological fidelity and computational efficiency.

3.4 • NEURAL CODING

In classical digital computing, information is represented by combining bits into richer structures, such as floating-point or integer numbers. For instance, the luminance of a pixel is typically stored as a discrete 8-bit or 32-bit integer. Conversely,

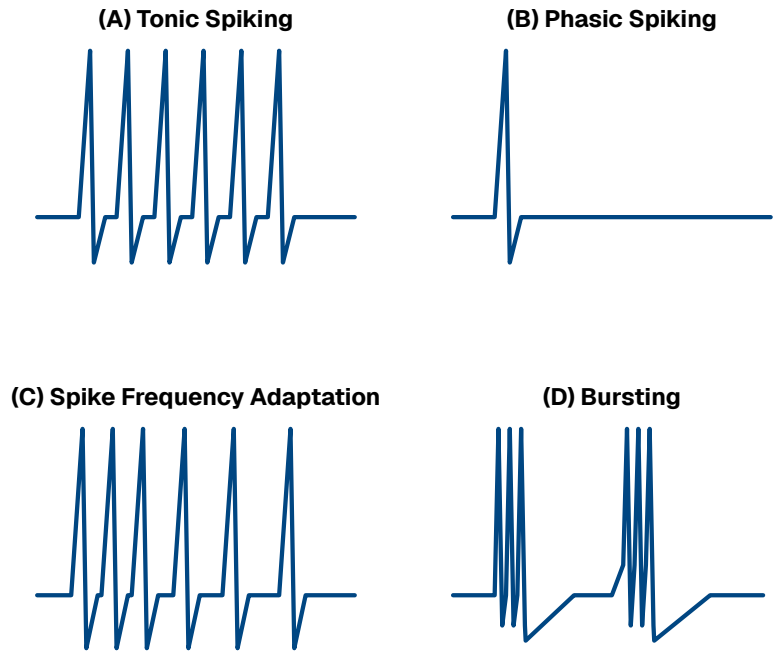

Figure 9: The Generalized Leaky Integrate and Fire (LIF) model is capable of reproducing the diverse firing patterns of biological cortical neurons, as categorized by Izhikevich (2003) [1].

analog electronics represent values as continuous currents or voltages, offering infinite resolution within the dynamic range of the hardware.

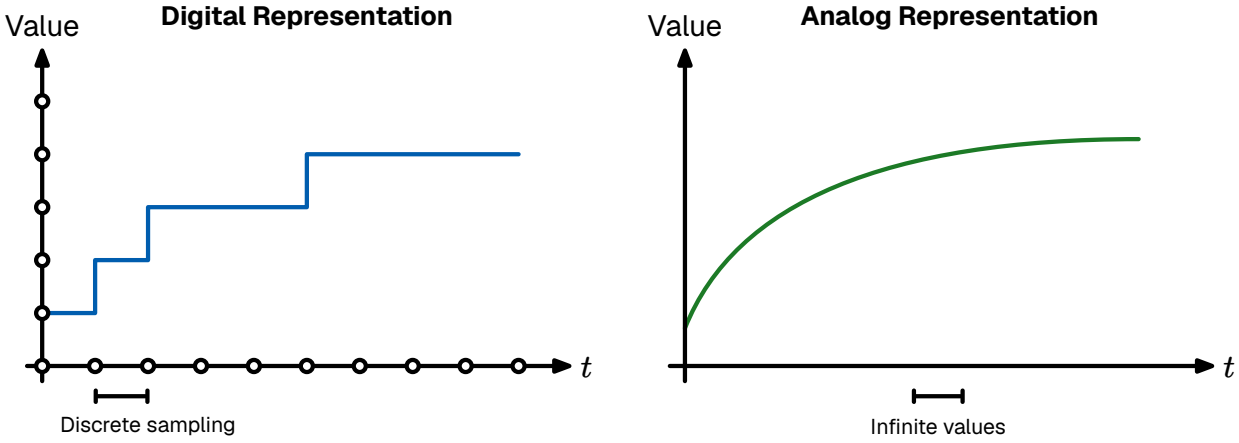

Figure 10: Digital left analog right representation

The biological brain occupies a unique middle ground. While neurons operate using analog membrane potentials, their communication output—the action potential—is discrete and binary. As established in Section 3.2, the waveform of a spike is stereotypical; it looks like a "digital bit" in amplitude. However, unlike a digital computer which is synchronized to a rigid clock, these spikes occur in continuous time. Therefore, the information in the nervous system is not stored in the shape of the signal, but in the structure of the spike train itself.

Deciphering the "Neural Code"—the set of rules by which sensory stimuli are translated into these spike sequences—remains one of the central challenges in neuroscience. Currently, several coding schemes are hypothesized to coexist, each offering different trade-offs between latency, information density, and robustness.

3.4.1 • RATE CODING

The most traditional interpretation of neural activity is rate coding. In this paradigm, information is conveyed by the mean firing frequency of a neuron over a specific temporal window. A strong stimulus (e.g., high pressure on skin) elicits a high firing rate, while a weak stimulus results in sparse activity.

This model effectively treats the neuron as an Analog-to-Digital converter where the precise timing of individual spikes is treated as noise; only the average count carries the signal. While rate coding is robust and easily observed in motor neurons, it suffers from a fundamental latency barrier. To estimate a rate with reasonable precision, the post-synaptic neuron must integrate spikes over a significant duration (tens or hundreds of milliseconds). This contradicts the rapid reaction times (often < 100 ms) observed in biological agents, suggesting that rate coding alone cannot account for time-critical processing.

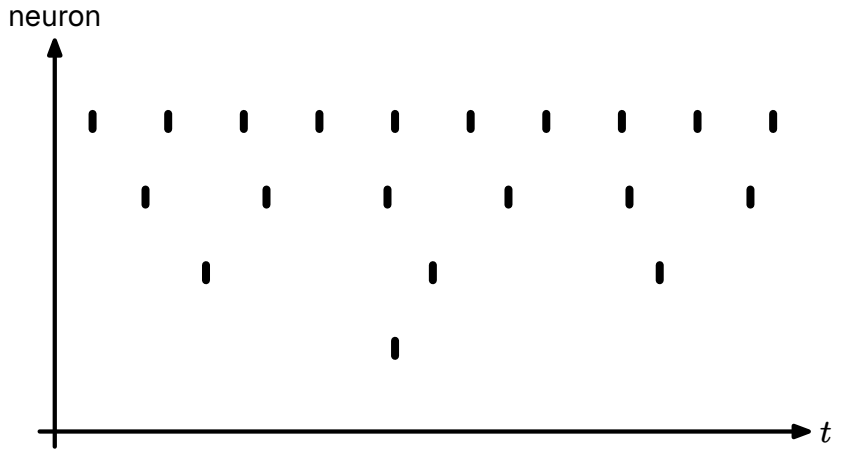

Figure 11: Rate Coding: The stimulus intensity is encoded in the frequency of the spike train. Stronger stimuli elicit more spikes per second.

3.4.2 • TEMPORAL CODING

To explain the speed of biological processing, neuromorphic engineering emphasizes temporal coding. In this regime, the precise timing of a spike carries significant information. A primary example is TTFS coding.

In a TTFS scheme, the intensity of a stimulus is inversely mapped to the latency of the response relative to a stimulus onset. A stronger input causes the neuron to integrate and cross the threshold faster, firing earlier than neurons receiving weak inputs. This shifts the computational model from counting spikes to a "race" between spikes.

In a network utilizing lateral inhibition (Winner-Take-All (WTA)), the first neuron to fire inhibits its neighbors, allowing a decision to be made as soon as the first meaningful bit of data arrives. This eliminates the need to wait for a time window to close, drastically reducing latency. Furthermore, since TTFS coding prioritizes the strongest signals, it acts as a natural filter: the most prominent features arrive

first, allowing the system to process signal over noise. This TTFS scheme forms the primary encoding modality for our MNIST experiments, as detailed in Section 5.3.1.

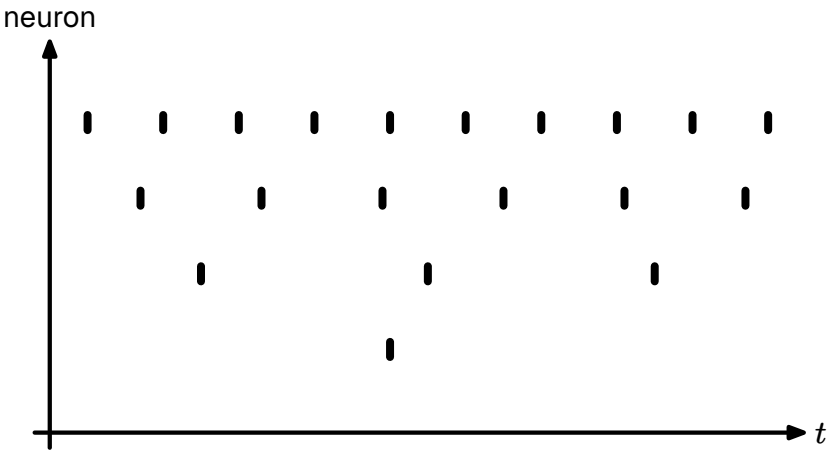

Figure 12: Temporal Codiing (Time-To-First-Spike (TTFS)): Stimulus intensity is encoded in the latency of the response. Stronger inputs (I_1) trigger an earlier spike (t_1) compared to weaker inputs (I_2).

3.4.3 • THE PHASE AMBIGUITY PROBLEM

A critical challenge in temporal coding is the need for a temporal reference frame. In Rate Coding, the "phase" (absolute timing) is irrelevant. However, in Temporal Coding, a spike at time t only has meaning relative to a reference t_0. If the receiver does not know when the stimulus started, it cannot decode the latency.

In engineering, this is solved by a global clock or a "frame start" signal. In the brain, evidence suggests that background oscillatory rhythms (brain waves, such as theta or gamma cycles) may serve as this global reference, allowing populations of neurons to synchronize their "clocks" and decode relative timings accurately.

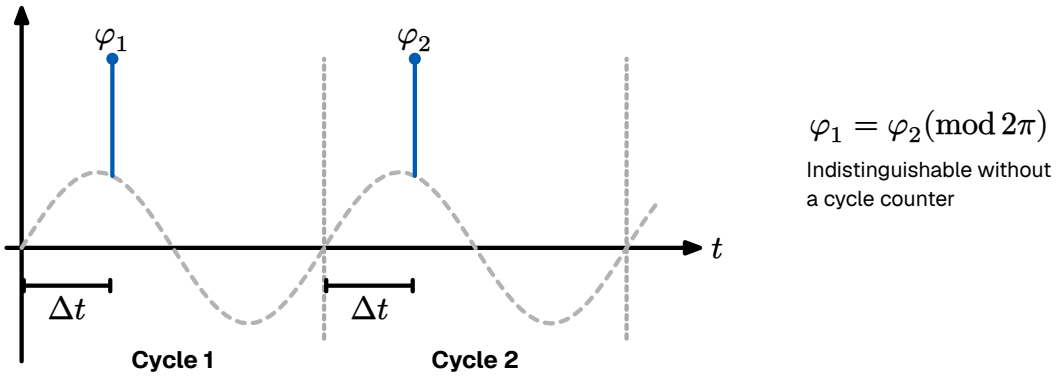

Figure 13: The phase ambiguity problem in temporal encoding. Spikes occurring at the same relative phase (φ_1 and φ_2) across different oscillation cycles are mathematically indistinguishable ($\varphi_1 = \varphi_2 (\mathrm{mod}\, 2\pi)$). Without a mechanism to track the global cycle count, downstream neurons cannot determine whether a spike represents a delayed response to a previous stimulus or an early response to a new one.

3.4.4 • POPULATION & SPARSE CODING

While single-neuron codes provide the basic signaling mechanism, the brain employs ensemble strategies to ensure robustness and precision. In population coding, variables are represented by the joint activity of a large group of neurons, each with broad, overlapping tuning curves. A classic example is found in the Primary Visual Cortex (V1), where orientation-selective neurons each respond maximally to a preferred angle but also fire weakly for nearby orientations. By reading the weighted population vector across the group, the network reconstructs the stimulus with far greater precision than any individual cell could provide alone. The brain further optimizes for metabolic efficiency through sparse coding, where only a small fraction of neurons are active at any moment. This strikes a mathematical balance between representational capacity and energy cost, and is naturally enforced by lateral inhibition circuits that suppress weaker, competing responses.

3.4.5 • COEXISTENCE OF CODES

These schemes are not mutually exclusive but complementary. A circuit may use TTFS for a rapid initial response — alerting the system to a salient change — before transitioning to rate-based activity for sustained processing. Neuromorphic systems often adopt this hybrid approach, using temporal codes for energy-efficient sparse event transmission and rate-based readouts for interfacing with downstream control systems. This thesis follows the same principle, using TTFS encoding for the transmission of visual features combined with a population-level representation at the hidden layer.

3.5 • NEURAL NETWORKS

Having established the mathematical description of the individual neuron, we now turn to the collective behavior of these units. A single neuron, regardless of its dynamical complexity, is of limited computational utility in isolation. Functional intelligence emerges only when these units are organized into specific structural topologies. We implement these topologies in our experimental framework, as detailed in Section 5.2.

The brain is not a random mesh of connections; it is constructed from recurring architectural "motifs" that appear across various cortical areas. Understanding these motifs is essential for designing neuromorphic systems that transcend simple feed-forward processing.

3.5.1 • SYNAPTIC EFFICACY & WEIGHTS

Before analyzing the structural topology of networks, we must define the fundamental unit of connectivity: the synapse. In the biological brain, neurons do not touch;

they are separated by a microscopic gap known as the synaptic cleft. Communication across this gap is chemical, mediated by the release of neurotransmitters.

The efficiency of this transmission—determined by factors such as the amount of neurotransmitter released and the number of post-synaptic receptors—is abstracted in mathematical models as the synaptic weight (w).

In the SNN formalism, the weight represents a scaling factor for the incoming spike. When a pre-synaptic neuron j fires a spike at time t_j, it induces a Post Synaptic Current (PSC) in neuron i scaled by the weight w_{ij}. Mathematically, the total synaptic input $I(t)$ is the weighted sum of all incoming spike trains:

$$I_{i(t)} = \sum_j w_{ij} \cdot S_{j(t)}$$

Equation 8: The synaptic input current as a weighted sum of incoming impulses.

Synaptic weights determine not just the magnitude but also if the synapse is excitatory or inhibitory.

- **Excitatory Synapses ($w > 0$):** These depolarize the target neuron, pushing its membrane potential closer to the firing threshold (e.g., Glutamate synapses).
- **Inhibitory Synapses ($w < 0$):** These hyperpolarize the target neuron, pushing the potential away from the threshold (e.g., GABA synapses).

A fundamental constraint in biological networks, known as Dale's Principle, states that a neuron performs the same chemical action at all of its synaptic outputs. This means a neuron is strictly excitatory or strictly inhibitory; it cannot send positive signals to one neighbor and negative signals to another. While standard ANNs often violate this rule for mathematical convenience (allowing weights to flip signs during training), bio-plausible neuromorphic architectures often enforce this constraint to mimic the distinct populations of Pyramidal (excitatory) and Interneuron (inhibitory) cells found in the cortex.

The network must maintain a precise Excitation-Inhibition (E/I) Balance. The brain operates at a critical point of instability:

- **Excess Excitation** leads to runaway feedback loops (analogous to epileptic seizures).
- **Excess Inhibition** leads to signal extinction (quiescence).

3.5.2 • DIRECTIONALITY

Structurally, neural topologies can be categorized by the flow of information.

In sensory peripheries (such as the retina) and early processing stages, information flows unidirectionally from input to output. This topology supports rapid, reflex-like feature extraction. This configuration is known as a feed-forward network, which is

mathematically equivalent to a Directed Acyclic Graph (Directional Acyclic Graph (DAG)) and serves as the standard architecture for most Deep Learning CNNs.

In higher cognitive areas, the dominant topology is recurrence. Neurons form feedback loops, connecting back to themselves or to distinct layers. This recurrence introduces a time component to the computation, transforming the network into a dynamical system where the current output depends not only on the input but on the network's previous state (history).

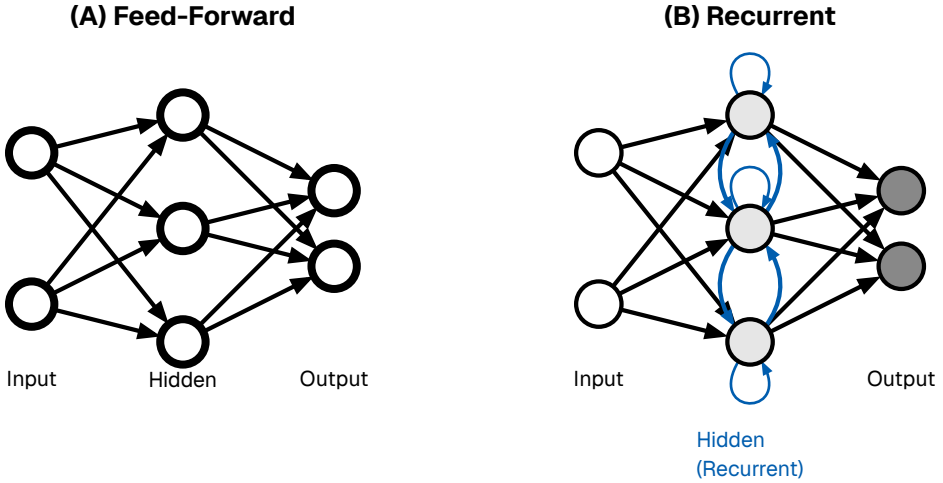

Figure 14: Network topologies. (A) Feed-Forward. (B) Recurrent.

3.5.3 • SYNAPTIC HYPOTHESIS: STRUCTURE AS FUNCTION

A foundational premise in neuromorphic engineering, derived from biological observation, is that the neuron operates largely as a generic processing unit. The functional identity of a neural circuit—whether it processes visual stimuli or governs motor control—is determined principally by the topology and efficacy of its synaptic interconnections.

This paradigm, known as the Synaptic Hypothesis, posits that the physical configuration of synaptic weights constitutes the substrate for all computation and memory. Unlike traditional Von Neumann architectures, where data is retrieved from a distinct memory module and processed in a central CPU, biological systems eliminate the distinction between "data" and "program." Memory is not a static artifact, but a latent computational potential distributed across the network's structural graph. Consequently, learning in a neuromorphic system is realized through the physical alteration of these synaptic weights, ensuring robust, decentralized processing that is inherently resistant to localized hardware failure (graceful degradation).

3.5.4 • INHIBITION PATTERNS

A ubiquitous micro-circuit motif in the cortex is lateral inhibition. In this configuration, an active excitatory neuron stimulates distinct inhibitory interneurons, which in turn suppress the activity of neighboring excitatory neurons. This compe-

tition engenders a WTA dynamic: as one neuron—representing a specific feature or decision—becomes active, it effectively silences its competitors. In the context of neuromorphic engineering, WTA circuits are indispensable; they provide a physical mechanism for both noise reduction, by actively suppressing weak, sub-threshold signals, and categorical decision making, enabling the circuit to autonomously select the most salient option without the need for a central processor to sort or compare values. We utilize this Winner-Take-All motif in our output layer (Section 5.3.3) to enforce categorical decision-making.

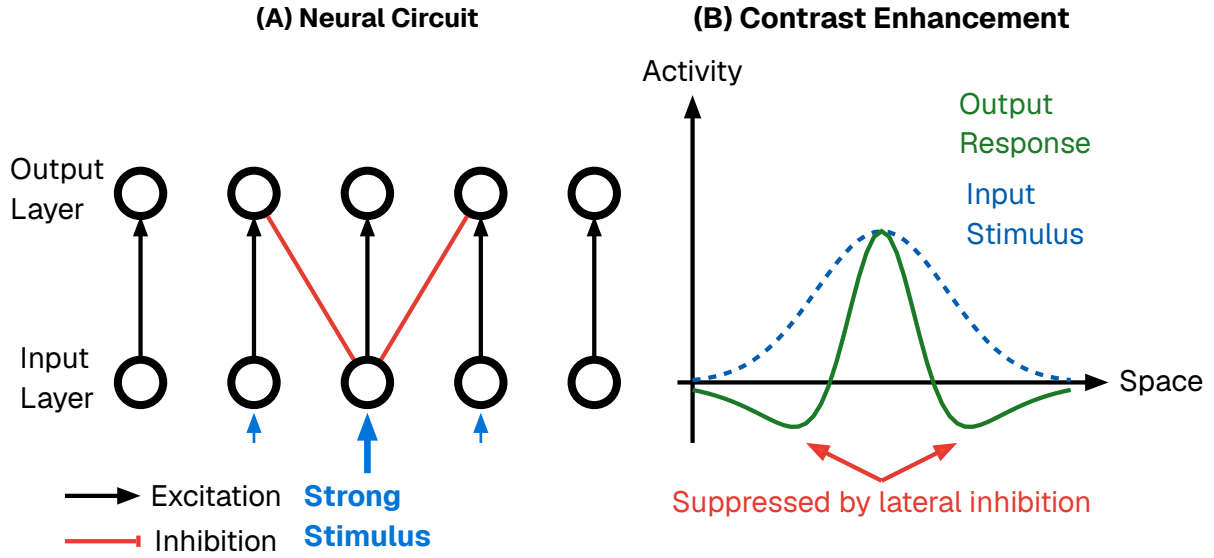

Figure 15: The mechanism of lateral inhibition. (A) A highly stimulated neuron in the input layer strongly excites its corresponding output neuron while simultaneously sending lateral inhibitory signals to its immediate neighbors. (B) This architectural motif acts as a spatial filter, producing a contrast enhancement effect. A broad input stimulus (dashed blue line) is transformed into a sharper output response (solid purple line) characterized by an amplified center and suppressed surroundings (a "Mexican hat" profile), thereby sharpening signal boundaries.

While lateral inhibition processes information in the spatial domain, Feed-Forward Inhibition (FFI) operates in the temporal domain. Structurally, this motif bifurcates an input signal into two parallel pathways: a direct excitatory route to the target neuron, and a disynaptic inhibitory route that reaches the same target with a slight synaptic delay. This architecture creates a narrow "temporal window of opportunity." Because the excitation triggers the neuron immediately before the delayed inhibition abruptly truncates the response, the neuron is prevented from integrating noise over extended durations. Consequently, FFI forces the neuron to function as a precise Coincidence Detector rather than a sluggish integrator, a dynamic that is fundamental to sound localization in the auditory cortex and fine-grain timing in the somatosensory system.

3.6 • BIOLOGICAL LEARNING

As previously established, the functional identity of a neural circuit is not defined by a transient software state, but by its physical hardware configuration. Consequently, "learning" in a biological substrate cannot be viewed as a simple parameter optimization; it is a fundamental morphological process. If structure dictates function, then the acquisition of new skills or memories necessitates the physical restructuring of the connectome itself.

Because the brain lacks a central supervisor or global communication bus, this restructuring must be driven by Locality. A synapse can only change based on information physically available at the cleft: the activity of the pre-synaptic axon, the voltage of the post-synaptic dendrite, and the immediate neurochemical environment. Despite this constraint, the brain successfully credits specific synaptic events with outcomes that occur seconds or minutes later.

This adaptation occurs across multiple timescales and spatial resolutions via two distinct mechanisms: Structural Plasticity (the rewiring of the network topology) and Synaptic Plasticity (the modulation of connection strength).

3.6.1 • STRUCTURAL PLASTICITY

While synaptic weight adjustment accounts for rapid learning and pattern recognition, the long-term storage of information and the optimization of energy efficiency are governed by structural plasticity. This mechanism involves the physical genesis (synaptogenesis) and removal (pruning) of synapses and even entire axonal branches.

- **Synaptogenesis**: When neurons are repeatedly co-active but lack a direct connection, the brain can physically grow new dendritic spines and axonal boutons to bridge the gap. This effectively alters the network's topology, creating new pathways for information flow where none existed before.
- **Pruning**: Equally critical is the removal of redundant or noisy connections. During sleep and developmental critical periods, the brain aggressively prunes weak synapses. This "sparsification" reduces metabolic cost and increases the signal-to-noise ratio of the circuit by removing irrelevant pathways.

In the context of the Synaptic Hypothesis, structural plasticity represents the "compiling" of temporary associations into permanent hardware architecture.

3.6.2 • SYNAPTIC PLASTICITY

Once a structural connection exists, its efficacy—the magnitude of the post-synaptic response to a pre-synaptic spike—must be tuned. In biological terms, this "weight" corresponds to the amount of neurotransmitter released and the density of receptors

on the receiving side. This fine-grained adjustment is governed by local learning rules.

3.6.3 • HEBBIAN LEARNING: RATE-BASED CORRELATION

The foundational axiom of biological learning was postulated by Donald Hebb in 1949. Hebb proposed that synaptic efficiency is a function of the correlated activity between two neurons. Colloquially summarized as "Neurons that fire together, wire together," this rule implies that the brain learns by detecting statistical regularities in sensory input.

Mathematically, if neuron A consistently takes part in firing neuron B, the connection from A to B is strengthened. This mechanism allows the brain to perform unsupervised clustering, physically encoding associations between features that occur simultaneously in the environment (e.g., the smell of smoke and the sight of fire).

3.6.4 • SPIKE-TIMING-DEPENDENT PLASTICITY (STDP)

Modern neuroscience has refined Hebb's macroscopic theory into a precise, millisecond-scale mechanism known as Spike-Timing-Dependent Plasticity (STDP). Unlike rate-based models, STDP operates on the precise timing of individual action potentials, introducing the critical element of causality.

The STDP rule adjusts the synaptic weight based on the relative timing difference (Δt) between the pre-synaptic input and the post-synaptic output:

- **Long-Term Potentiation (LTP)**: If the input spike arrives **before** the output spike ($\Delta t > 0$), it implies the input contributed to the firing. The synapse is strengthened to reinforce this causal link.
- **Long-Term Depression (LTD)**: If the input spike arrives **after** the output spike ($\Delta t < 0$), the input was irrelevant to the decision. The synapse is weakened.

This asymmetry allows the network to self-organize, naturally filtering out random noise while reinforcing specific spatiotemporal patterns. We adapt this causal rule to a discrete temporal window for our unsupervised learning experiments in Section 5.4.6.

3.6.5 • HOMEOSTATIC PLASTICITY

If Hebbian mechanisms (LTP) were the sole drivers of plasticity, neural networks would be inherently unstable. A positive feedback loop would emerge where strengthened synapses drive higher firing rates, which in turn induce further strengthening, leading to runaway excitation (seizures). Conversely, unchecked LTD could silence a network entirely.

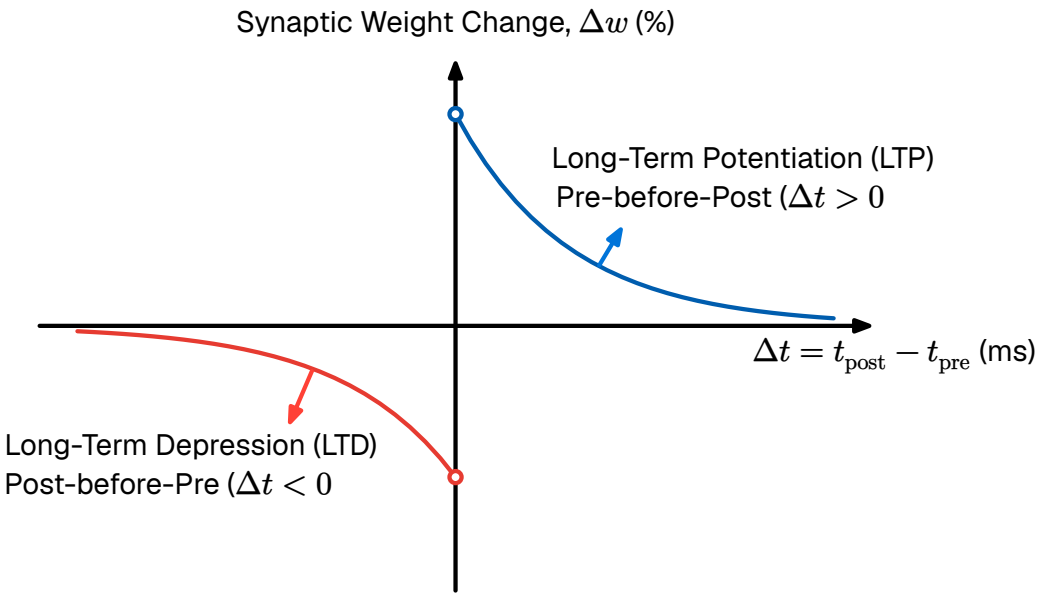

Figure 16: The Spike-Timing-Dependent Plasticity (STDP) Learning Curve. Synaptic weight change is plotted against spike timing difference. Pre-before-post timing triggers strengthening Long-Term Potentiation (LTP), while post-before-pre triggers weakening Long-Term Depression (LTD).

To maintain stability, the brain employs Homeostatic Plasticity (or Synaptic Scaling). This is a global regulatory mechanism that operates on a slower timescale (minutes to hours). It functions as a negative feedback loop: if a neuron's average firing rate exceeds a target set-point, the cell chemically downscales the strength of all its incoming synapses. This ensures that neurons remain within a sensitive dynamic range, preventing saturation regardless of how strong the inputs become.

The following chapter shifts perspective—from biology to engineering. We examine the mathematical framework of modern Deep Learning, to identify where its abstractions diverge from biological reality and what computational cost those divergences impose. This analysis will make explicit the bottlenecks that neuromorphic architectures are designed to resolve, grounding the methodological choices of this thesis in a concrete technical rationale.

4 • TECHNICAL DETAILS OF MACHINE LEARNING

This chapter delineates the technical foundations of modern artificial intelligence, contrasting the established paradigms of Deep Learning (DL) with the emerging principles of Neuromorphic Engineering. We begin by analyzing the algorithmic architecture of standard Deep Learning, identifying the computational bottlenecks inherent in its reliance on dense matrix multiplication and backpropagation.

A critical distinction must be drawn between biological plausibility and bio-inspired engineering. From an engineering perspective, the primary objective is functional utility. An engineer may treat the brain merely as a source of heuristic inspiration rather than a blueprint to be copied dogmatically. However, the pursuit of biologically plausible systems remains vital; it offers potential advantages in robustness and energy efficiency while serving as a verification tool for neuroscience.

4.1 • OPTIMIZATION

Optimization is the selection of a "best candidate" with regard to defined criteria. Biological learning fits this description, where the optimal candidate is the configuration of synaptic weights that performs well for a specific task. Therefore, it is useful to establish a mathematical framework for this process.

Fundamentally, a deep learning model operates as a function approximator. We assume the existence of an unknown underlying function $f : X \to Y$ that perfectly maps inputs to their target outputs. Since this true function is unknown, we construct a family of hypothesis functions $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ to approximate it. Here, $\boldsymbol{\theta} \in \mathbb{R}^d$ represents the state of the system—a vector containing all tunable parameters, such as synaptic weights or biases. The dimensionality d corresponds to the degrees of freedom of the model.

The key problems in optimization are defining the objective goal (the loss function) and finding the parameter configuration $\boldsymbol{\theta}$ that achieves that goal.

4.1.1 • SUPERVISED LEARNING

To guide the search for optimal parameters $\hat{\boldsymbol{\theta}}$, we must quantify the divergence between the model's predictions and the ground truth. We define a scalar Loss Function $\mathcal{L}(\hat{\boldsymbol{y}}, \boldsymbol{y})$ that evaluates the error on a single data point. To ensure generalization, we seek to minimize the Empirical Cost Function $J(\boldsymbol{\theta})$, defined as the average loss over a dataset of size N:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), \boldsymbol{y}_i)$$

Geometrically, the cost function $J(\boldsymbol{\theta})$ induces an Optimization Landscape. Finding a low-energy state in this non-convex topology is the central challenge of AI training. We rely on iterative optimization algorithms, principally Gradient Descent. This method updates the system state in the direction opposite to the gradient vector $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ (the steepest ascent). The update rule for iteration t is:

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)$$

Here, η represents the Learning Rate. Because computing the gradient over the entire dataset N is computationally prohibitive, modern AI employs Stochastic Gradient Descent (SGD), approximating the gradient using small random subsets (mini-batches). This introduces beneficial noise, preventing the system from getting trapped in shallow local minima.

Crucially, gradient descent requires the loss function to be differentiable. As will be discussed later, this presents a significant challenge for optimizing neuromorphic systems utilizing discrete, non-differentiable spike trains.

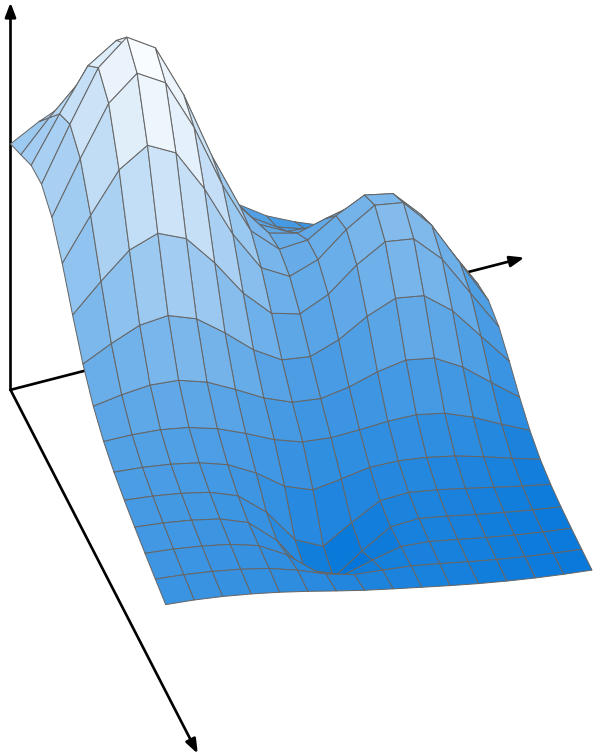

Figure 17: The Optimization Landscape. The system seeks to traverse the high-dimensional surface defined by $J(\boldsymbol{\theta})$ to find the global minimum $\boldsymbol{\theta}^*$, using the gradient ∇J as a navigational compass.

Strictly minimizing the empirical cost carries the risk of overfitting — the model memorizes training data including noise rather than learning the underlying function. In biological systems this is naturally regulated by metabolic constraints; the

brain prunes weak connections to maintain a sparse topology, effectively trading model complexity for generalization. In artificial systems this is managed via explicit regularization penalties added to the cost function.

4.1.2 • UNSUPERVISED LEARNING

While supervised learning relies on labeled targets, biological systems predominantly learn via Unsupervised Learning. In this regime, the dataset consists only of input vectors $X = \{\boldsymbol{x}_1, ..., \boldsymbol{x}_N\}$. The optimization objective shifts from minimizing prediction error to minimizing representation error.

Mathematically, the goal is often to discover a lower-dimensional manifold that efficiently captures the structure of the data. A common formulation is the minimization of Reconstruction Loss, where the system attempts to compress the input into a latent code and reconstruct it:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \| \boldsymbol{x}_i - f_{\text{decode}\left(f_{\text{encode}(\boldsymbol{x}_i;\boldsymbol{\theta})}\right)} \|^2$$

Alternatively, the system may optimize for clustering density or distances between feature centroids. The distinction between supervised and unsupervised learing is critical for Neuromorphic Engineering, as biological plasticity rules (like STDP) are unsupervised, functioning by detecting statistical correlations in the input stream to build internal representations without external labels.

4.2 • DEEP LEARNING FRAMEWORK

Modern Deep Learning aggregates simple units into high-dimensional layers. A deep network with L layers is expressed as a composite function mapping input $\boldsymbol{x}$ to output $\boldsymbol{y}$ through nested operations:

$$\boldsymbol{y} = f_L(...f_2(f_1(\boldsymbol{x})))$$

During the Forward Pass, each layer performs an Affine Transformation (a linear rotation and scaling of data via weight matrix $\boldsymbol{W}$ and bias $\boldsymbol{b}$) followed by a Non-Linear Activation (σ):

$$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)} \boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}$$

$$\boldsymbol{a}^{(l)} = \sigma\left(\boldsymbol{z}^{(l)}\right)$$

The non-linearity prevents the deep stack from collapsing into a single linear equation. Modern networks rely on the Rectified Linear Unit (ReLU), $f(x) = \max(0, x)$. Its derivative (0 or 1) preserves the magnitude of the gradient, allowing error signals to travel through deep structures without vanishing.

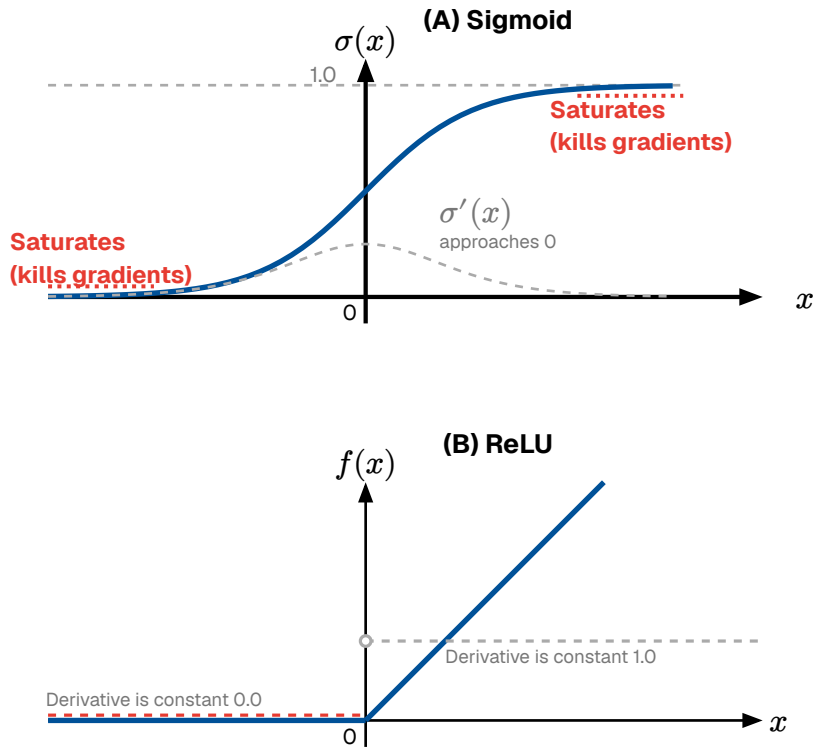

Figure 18: Activation Functions. The Sigmoid (left) saturates gradients. The ReLU (right) preserves gradient magnitude for positive inputs.

During the Backward Pass, Backpropagation recursively applies the Chain Rule via Automatic Differentiation to attribute the total error $J(\boldsymbol{\theta})$ to specific weights.

To achieve high throughput, these operations are vectorized. The affine transformation for an entire layer is executed as a Dense Matrix Multiplication. This is the defining characteristic of modern AI hardware. A deep network is effectively a sequence of massive matrix multiplications, which is highly parallelizable on GPUs

$$\begin{bmatrix} w & w & w & w \end{bmatrix} \begin{bmatrix} x \\ x \\ x \\ x \end{bmatrix} \qquad \begin{bmatrix} w & w & w & w \\ w & w & w & w \\ w & w & w & w \\ w & w & w & w \\ w & w & w & w \\ w & w & w & w \end{bmatrix} \begin{bmatrix} x \\ x \\ x \\ x \end{bmatrix}$$

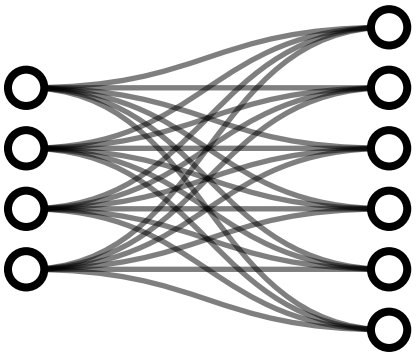

Figure 19: Deep Learning as Matrix Multiplication. Forward and backward passes rely on dense matrix products, necessitating high-bandwidth memory access.

4.2.1 • CONVOLUTIONAL NEURAL NETWORKS (CNNS)

For visual tasks, standard Multi-Layer Perceptrons scale poorly; connecting every pixel to every neuron ignores the spatial structure of the data and creates an intractable number of weights. To solve this, DL utilizes CNNs.

CNNs apply small, learnable weight matrices known as "kernels" or "filters" that slide (convolve) across the input image. This architecture introduces two critical inductive biases:

1. **Local Connectivity:** Neurons only process a small, local receptive field, analogous to the biological visual cortex.
2. **Weight Sharing:** The exact same kernel is applied across the entire image, drastically reducing the number of tunable parameters and establishing translation invariance (a feature learned in one corner of an image can be recognized anywhere else).

While CNNs are the standard baseline for spatial processing, they remain fundamentally synchronous and frame-based, evaluating the entire image structure in dense mathematical passes regardless of local activity.

4.3 • WHY IS DEEP LEARNING INEFFICIENT?

While the matrix-centric formulation of Deep Learning enables high-throughput parallelization on GPUs, it fundamentally conflicts with the physical constraints of modern computing hardware. As models scale to billions of parameters, the primary bottleneck shifts from algorithmic capability to physical realizability. This inefficiency manifests in four distinct engineering dimensions:

4.3.1 • THE VON NEUMANN BOTTLENECK & DATA MOVEMENT

The most significant physical limitation is the Von Neumann Architecture, which physically separates the Processing Unit from the Memory Unit. To perform a single inference step, the processor must fetch the entire weight matrix from off-chip DRAM to on-chip registers, perform the calculation, and write the results back.

According to Horowitz and Dally [1], fetching a 32-bit value from off-chip DRAM consumes approximately 640 pJ, whereas performing a floating-point addition on that value consumes only 0.1 pJ. The system expends 99.9% of its energy transporting data, and only 0.1% actually computing.

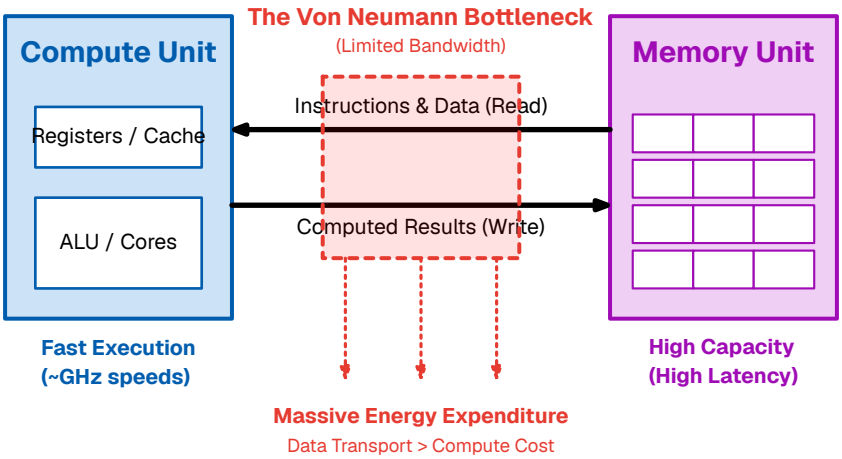

Figure 20: The Von Neumann Bottleneck. The separation of memory and compute forces massive energy expenditure on data transport.

4.3.2 • DENSE PROCESSING OF SPARSE DATA

Standard Deep Learning implementations rely on Dense Matrix Multiplication (GEMM). This approach is algorithmically rigid: it executes the same number of operations regardless of the data content.

Real-world sensory data is often highly sparse, and the ReLU activation function naturally produces activation maps where the majority of values are zero. However, a standard GPU is "blind" to this sparsity. It will dutifully fetch a zero from memory and multiply it by a weight ($0 \times w = 0$), consuming energy and clock cycles to produce a null result. Deep Learning's inability to exploit this silence represents a massive structural inefficiency.

4.3.3 • THE HIGH COST OF SYNCHRONY

Deep Learning hardware is typically Synchronous, operating in lockstep with a global clock. Driving a high-frequency clock signal across an entire silicon die forces billions of transistors to charge and discharge continuously, regardless of whether the chip is doing useful work. In high-performance processors, this clock distribution network alone can consume 30% to 40% of the total power budget. Furthermore, global synchronization enforces a "worst-case" latency: faster computations must sit idle and wait for the slowest operations to finish before the next clock cycle begins.

4.3.4 • BACKPROPAGATION AND GLOBAL DEPENDENCIES

Finally, Backpropagation imposes severe constraints on memory and latency because it is non-local in both time and space. To update a specific weight, the system must wait for the Forward Pass to finish, calculate the global error, and wait for the backward pass to propagate the gradient.

This creates a "Locking Problem." The activations of every intermediate layer must be stored in high-speed memory (VRAM) for the duration of the entire pass, preventing that memory from being reused. Additionally, a local synapse cannot adapt to local changes instantly; it is shackled to the global error loop.

4.4 • PRINCIPLES OF NEUROMORPHIC ENGINEERING

As established in the *History & Developments* chapter, Neuromorphic Engineering is the translation of biological dynamics into silicon hardware. It replaces the rigid, clock-driven logic of standard computing with the adaptive, event-driven dynamics of neural tissue. This approach rests on three architectural pillars that directly address the bottlenecks of Deep Learning:

- **Co-location of Memory and Compute (The Synaptic Principle):** Neuromorphic architectures eliminate the Von Neumann bottleneck by distributing memory across the silicon die. Each artificial neuron stores its own state and synaptic weights locally, processing data **in situ** to eliminate the energy cost of shuttling data.
- **Event-Driven Asynchrony (The Action Potential Principle):** Neuromorphic systems abandon the global clock. They operate asynchronously, driven strictly by the arrival of data. If a part of the network is not processing information, it consumes negligible power, ensuring energy scales linearly with task complexity rather than network size.
- **Sparse Communication (The Spike Principle):** Neuromorphic systems utilize binary Spikes for communication. Information is encoded in the precise timing of events rather than complex magnitudes, drastically reducing the bandwidth required to transmit information between neurons.

4.5 • TRAINING SPIKING NETWORKS

While the physical architecture of neuromorphic systems is highly efficient, training these networks presents a fundamental mathematical challenge. Standard deep learning relies on gradient descent, but backpropagation cannot be directly applied to native SNNs.

In a spiking network, the neuron's activation function is a discontinuous step function (the Dirac delta event threshold). The derivative of this function is zero everywhere except at the exact moment of the spike, where it is undefined. Consequently, gradients calculated using the chain rule immediately drop to zero—known as the "Dead Neuron" problem—preventing error signals from flowing backward through the network to update the weights.

To circumvent this non-differentiability and optimize network parameters, the field of neuromorphic engineering generally employs two distinct paradigms:

4.5.1 • DIRECT WEIGHT TRANSFER

A pragmatic engineering approach to bypass the dead neuron problem is offline training. In this paradigm, a standard, continuous ANN (such as a network utilizing ReLU activations) is trained conventionally using backpropagation. Once convergence is achieved, the learned weights are directly mapped onto a structurally identical Spiking Neural Network.

The underlying premise is that the continuous activation values of the ANN can be approximated by the discrete firing rates of the SNN over a set time window. While this method allows the spiking system to inherit the high accuracy of gradient descent, direct weight transfer requires careful scaling and normalization. If the weights are copied without adjustment, the resulting SNN may suffer from catastrophic saturation (firing constantly) or severe signal degradation (failing to reach the spiking threshold). This conversion process and its associated quantization constraints are evaluated in our implementation in Section 5.4.5.

4.5.2 • NATIVE LOCAL LEARNING (STDP)

To fully exploit the energy efficiency and event-driven dynamics of neuromorphic hardware, training must ideally occur natively on the spiking substrate. This requires abandoning global backpropagation in favor of biologically plausible, mathematically local learning rules.

As established in Section 3.6, Spike-Timing-Dependent Plasticity (STDP) adjusts synaptic weights based strictly on the temporal correlation of local pre- and post-synaptic spikes. Because STDP relies exclusively on local physical events rather than global error gradients, it does not require a differentiable loss function. This allows the network to completely bypass the dead neuron problem, enabling unsupervised feature extraction and real-time adaptation directly on the spiking architecture.

4.5.3 • SURROGATE GRADIENT DESCENT

For completeness, it must be noted that the current dominant paradigm in SNN research utilizes Surrogate Gradients. In this approach, the network operates using the discontinuous spike step-function during the forward pass, but temporarily replaces the undefined derivative with a smooth, continuous approximation (a "surrogate") during the backward pass. While this thesis focuses on evaluating direct weight transfer and native unsupervised STDP, surrogate methods represent a highly effective hybrid approach, allowing backpropagation-like algorithms to estimate gradients across discrete spiking layers.

4.6 • NEUROMORPHIC HARDWARE TECHNIQUES

Central to realizing these computational efficiencies in physical hardware is Address Event Representation (AER), a communication protocol that mirrors the sparse nature of biological spikes. Instead of continuous data streaming, the hardware only transmits the "address" of a firing neuron across a shared digital bus, allowing a single physical wire to represent thousands of virtual axonal projections.

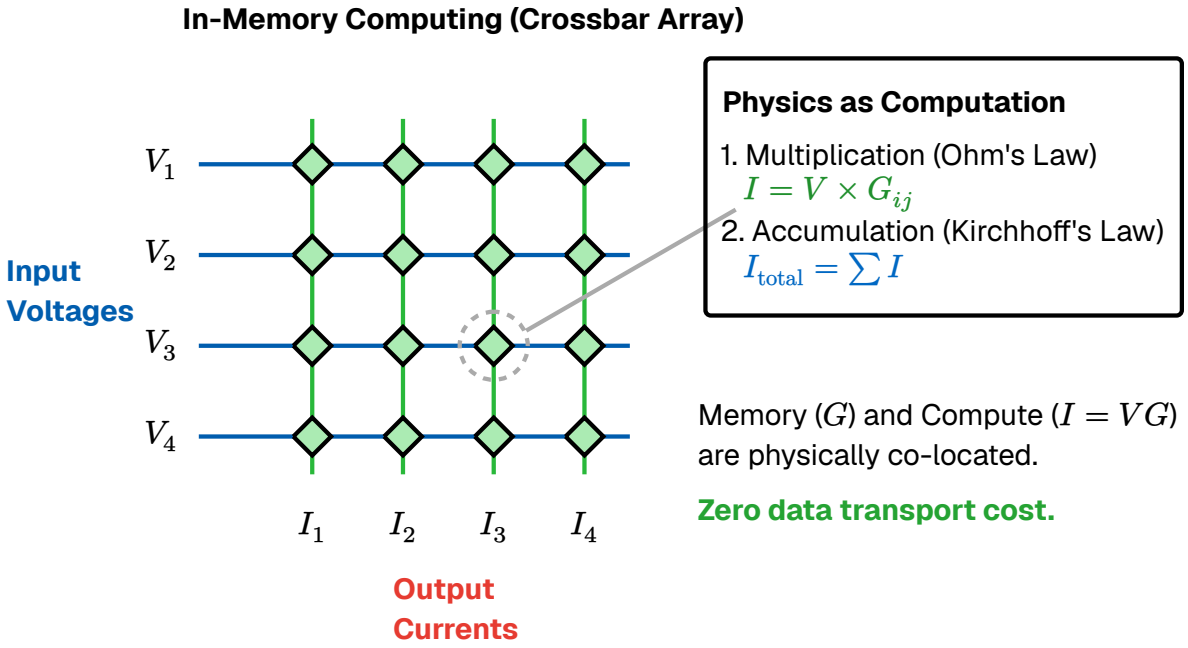

Figure 21: In-Memory Computing via a Crossbar Array. Unlike von Neumann architectures, memory and computation are physically co-located. Input voltages (V) are applied to the wordlines. Memory elements at the junctions hold programmable conductances (G). Multiplication is natively performed at each junction by Ohm's Law ($I = V \times G$), and resulting currents are summed along the bitlines via Kirchhoff's Current Law. This allows dense matrix-vector multiplications to occur in a single analog time step with zero data transport cost.

The crossbar array provides a direct structural surrogate for the neural neuropil. Because the architecture handles multiplication and summation natively through physical laws, it is uniquely suited to implement biological "macro-motifs." By routing bitline currents through local feedback loops, the hardware can instantiate complex dynamics such as Lateral Inhibition and Winner-Take-All circuits without the overhead of high-level software instructions.

This synergy between physical topology and functional motifs allows the hardware to inherit the computational efficiency of the neocortex, effectively making the architecture itself the algorithm.

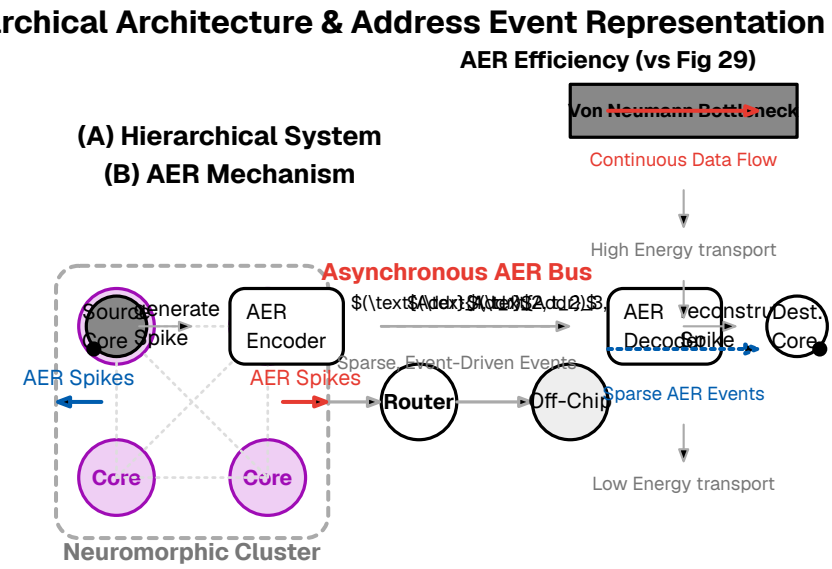

Figure 22: Architectural Comparison. (Left) The Von Neumann architecture separates memory and compute, creating a bottleneck. (Right) The Neuromorphic architecture co-locates them, mimicking the distributed topology of biological neural networks.

Having established the theoretical limitations of traditional deep learning and the physical principles of neuromorphic engineering, the following chapter details the specific methodologies, network architectures, and software frameworks utilized in this thesis to evaluate ANN-to-SNN weight conversion and native STDP learning on visual event data.

5 • METHOD

This chapter details the specific implementations of the neuromorphic architectures proposed to address the limitations of standard deep learning. Aligning with the biological constraints of sparsity, asynchrony, and locality established in previous chapters, we outline the construction and evaluation of a SNN.

To empirically validate the theoretical advantages of neuromorphic algorithms, we evaluate the system on a benchmark image classification task. The experiment is bifurcated into three distinct phases:

1. **Neuron Model Evaluation:** Evaluating the decoding efficiency and accuracy of different simulated spiking models, specifically comparing a biologically inspired Leaky Integrate-and-Fire (LIF) model against a computationally streamlined Current-Accumulating (Ramp) model.
2. **Inference via Weight Transfer:** Evaluating the zero-shot performance of these SNNs initialized with weights directly mapped from a classically trained Artificial Neural Network (ANN).
3. **Native Unsupervised Learning:** Training the SNN from scratch utilizing local STDP.

5.1 • DATASET & PRE-PROCESSING

To benchmark these algorithms, we require a dataset that necessitates the extraction of complex spatial features but remains computationally tractable for rapid experimental iteration. We utilize the MNIST database of handwritten digits [1].

The dataset consists of a training set of 60,000 examples and a test set of 10,000 examples of digits (0-9). Each instance is a 28×28 pixel grayscale image. While standard deep learning models routinely score over 90% accuracy on this task, making it largely a solved problem in classical AI, its well-understood feature space makes it an ideal, isolated baseline. Because the spatial hierarchy of MNIST is relatively shallow, it allows us to evaluate the efficacy of neuromorphic learning rules without the confounding variables introduced by massive, multi-layered convolutional architectures.

Crucially, the MNIST images are pre-processed by the dataset creators to be size-normalized and centered within the pixel grid using the center of mass of the pixels. This spatial alignment is a vital prerequisite for our chosen network topology. Unlike CNNs, which slide localized filters across an image, the FCN utilized in this thesis lacks translation invariance. If a digit were shifted several pixels off-center, the FCN would perceive it as an entirely novel pattern. The pre-centered nature of MNIST mitigates this limitation, ensuring that the network can reliably map specific geometric strokes to specific input neurons.

Furthermore, the dataset exhibits a high degree of spatial sparsity. In a typical MNIST image, the vast majority of pixels represent the empty background. From a neuromorphic engineering perspective, this sparsity is highly advantageous. As established in the theoretical framework, event-driven systems expend energy strictly when events occur. A sparse input array ensures that the majority of input neurons remain quiescent, minimizing bus congestion and validating the energy-efficiency claims of the proposed SNN.

Before the raw images can be converted into temporal spike trains, they must undergo standard spatial pre-processing to ensure compatibility with the network's mathematical boundaries. This consists of two primary transformations:

1. **Normalization**: Raw pixel intensities in the MNIST dataset range from 0 (pure black) to 255 (pure white). To stabilize the learning algorithms and ensure consistent weight scaling, these values are strictly normalized to a continuous float range of $p_i \in [0.0, 1.0]$.
2. **Flattening**: Because this thesis utilizes a Fully Connected Network (FCN) to facilitate direct weight transfer, the 2D spatial structure of the images must be unrolled. Each 28×28 matrix is flattened into a 1-dimensional vector of 784 elements.

Consequently, every individual image is presented to the system as a discrete array of 784 normalized intensities. In the classical Artificial Neural Network (ANN), these continuous values are fed directly into the input neurons. However, because SNNs operate exclusively on discrete events, these normalized values must be passed through a temporal encoding algorithm before inference or learning can begin.

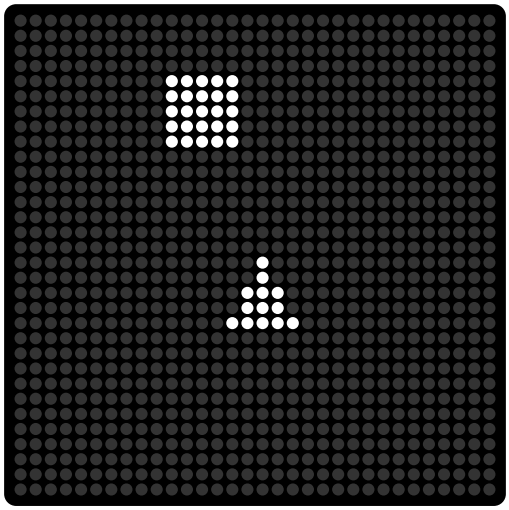

Figure 23: Sample of the MNIST dataset. The 28x28 images are normalized and flattened into 1D vectors before being translated into temporal spike events.

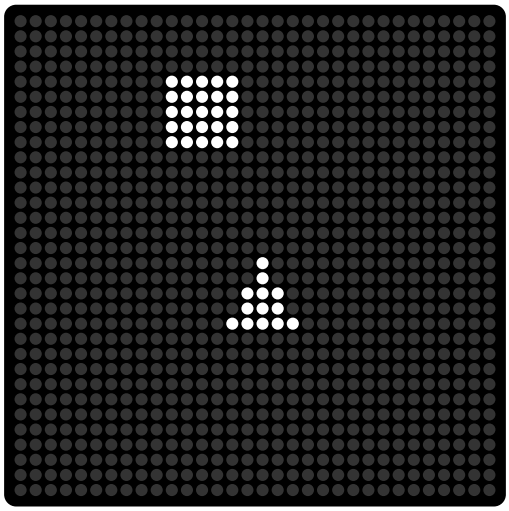

Figure 24: Sample of the MNIST dataset. The 28x28 images are normalized and flattened into 1D vectors before being translated into temporal spike events.

5.2 • NETWORK ARCHITECTURE

As established in Section 3.5, functional intelligence emerges from structural topology. To facilitate a direct, one-to-one mapping of synaptic weights from the offline ANN to the native SNN, both models must share an identical macroscopic topology. Therefore, this implementation utilizes a FCN, also known as a MLP, rather than a CNN.

While CNNs are the standard baseline for vision tasks due to their spatial inductive biases, transferring convolutional kernels into a spiking substrate introduces significant mapping complexities—specifically the need to physically unroll and duplicate shared weights across the spiking array. An FCN provides a straightforward, mathematically transparent architecture for cleanly evaluating direct weight transfer and STDP without confounding architectural variables.

The network is structured as a shallow hierarchy to capture the primitive geometric features of the dataset. Let N_l denote the number of neurons in layer l. The formal architecture is defined as follows:

- **Input Layer (L_0):** The 28×28 pixel grayscale images are flattened into a 1D vector, requiring $N_0 = 784$ input neurons.
- **Hidden Layer (L_1):** A fully connected intermediate layer consisting of $N_1 = 128$ neurons. The synaptic connections are defined by the weight matrix $W^{\{(1)\}} \in \mathbb{R}^{\{N_1 \times N_0\}}$.
- **Output Layer (L_2):** $N_2 = 10$ neurons, corresponding directly to the categorical digit classes (0 through 9). The connections from the hidden layer are defined by the weight matrix $W^{(2)} \in \mathbb{R}^{\{N_2 \times N_1\}}$.

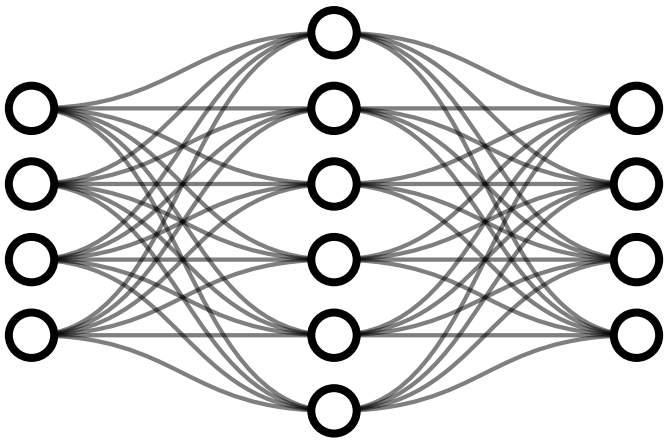

Figure 25: Network architecture. The 28x28 images are flattened and passed through a Fully Connected Network. Both the Artificial Neural Network (ANN) and Spiking Neural Network (SNN) share this identical macroscopic topology.

```
  run_snn_pipeline(config) → float:
1   dataset = load_and_preprocess_mnist()
2   W1, W2 = initialize_network_weights()
3   if config.mode == “PHASE_II_WEIGHT_TRANSFER”:
4       W1, W2 = load_pretrained_ann_weights()
5       W1, W2 = quantize_and_scale_to_int8(W1, W2)
6       accuracy = evaluate_network(dataset.test, W1, W2, config.T_max)
7       return accuracy
8   else if config.mode == “PHASE_III_UNSUPERVISED_STDP”:
9       for epoch in range(config.epochs):
10          for image in dataset.train:
11              spikes_in = encode_ttfs(image, config.T_max)
12              spikes_out = simulate_saccade_wta(spikes_in, W1, W2,
                config.T_max)
13              W1, W2 = parallel_apply_stdp(spikes_in, spikes_out, W1, W2)
14              W1 = enforce_homeostatic_normalization(W1)
15      freeze_synaptic_plasticity()
16      assign_post_hoc_labels(dataset.train_subset, W1, W2)
17      accuracy = evaluate_network_with_assigned_labels(dataset.test, W1,
        W2)
18      return accuracy
```

Algorithm 1: Overarching Program Flow: Spiking Neural Network (SNN) Simulation Pipeline

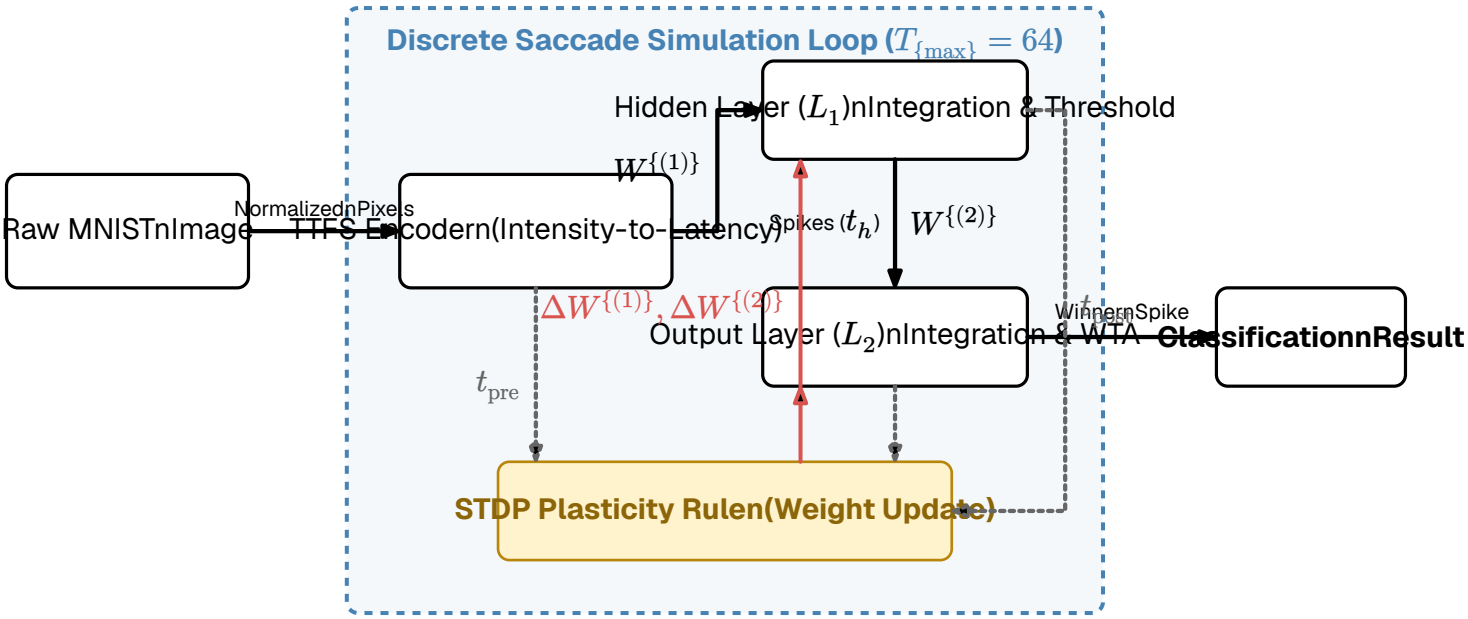

Figure 26: Software architecture and data flow diagram of the spiking neural network simulator. The diagram illustrates the pipeline from raw MNIST images to temporal spikes, the core event-driven simulation loop, and the local plasticity feedback mechanism.

5.2.1 • WEIGHT INITIALIZATION AND BIASES

For the baseline ANN, the weight matrices are initialized using standard PyTorch defaults to ensure stable variance during the initial forward passes.

A critical architectural consideration in ANN-to-SNN conversion is the handling of bias vectors ($b^{(l)}$). While standard ANNs utilize biases to shift activation functions, representing a static continuous bias in a spiking network requires either injecting a constant background current into the neuron or actively modifying the neuron's firing threshold. To maintain pure, event-driven sparsity and isolate the performance of the synaptic weights, explicit bias terms were omitted entirely from both architectures.

5.2.2 • ANN-SNN COMPATIBILITY

Despite the macroscopic symmetry required for weight sharing, the microscopic dynamics of the two networks differ fundamentally. The offline ANN utilizes standard continuous activation functions (specifically ReLU) to compute smooth gradients during backpropagation.

In contrast, the SNN replaces these continuous functions with Integrate-and-Fire neurons governed by a strict voltage threshold. This simulates the biological "all-or-nothing" action potential, acting as a hard step function. Furthermore, the spiking architecture utilizes lateral inhibition at the output layer. This engenders a WTA dynamic: as the network integrates evidence over time, the first output neuron to reach its threshold heavily suppresses its competitors, forcing a definitive categorical decision and actively filtering sub-threshold noise.

```
simulate_saccade(image_data, W1, W2, T_max):
    spikes_in = encode_ttfs(image_data)
    reset_neuron_states(V_m1, V_m2)
    for t from 0 to T_max:
        fired_L1 = forward_pass_optimized(spikes_in, W1, V_th1)
        fired_L2 = forward_pass_optimized(fired_L1, W2, V_th2)
        if any(fired_L2):
            winner = select_first(fired_L2)
            return winner, t
    return None, T_max
```

Algorithm 2: The Discrete Time-Step (Saccade) Simulation Loop

The inference execution is governed by a discrete "saccade" simulation loop. The temporal window for a single MNIST image is strictly bounded to $T_{\max} = 64$ discrete time steps (ticks).

To ensure that deep layer spikes have sufficient time to propagate through the network before the simulation window closes, the input encoding is temporally padded. Using the linear intensity-to-delay algorithm defined in Section 3.3.1, the maximum input delay is capped at $t = 32$. Therefore, a pure black pixel spikes at $t = 32$, leaving 32 subsequent ticks for the hidden and output layers to integrate the final data.

At each time step $t \in [0, 64)$, the simulator executes a parallelized Window Integrator evaluation:

1. **Event Detection:** The simulator queries all input and hidden neurons to identify which neurons are scheduled to spike at the exact current tick t.
2. **Parallel Integration:** Using optimized matrix-vector multiplication (`torch.mv`), the synaptic weights of the active neurons are concurrently summed into the membrane potentials (V_m) of the downstream layer.
3. **Thresholding & Single-Spike Constraint:** A neuron in the hidden layer ($N_1 = 128$) fires if its potential crosses a predefined high threshold ($V_{\text{th\_h}}\} = 200.0$). The output layer ($N_2 = 10$) operates on a lower threshold ($V_{\text{th\_o}}\} = 100.0$).

Crucially, to enforce the Time-to-First-Spike (TTFS) paradigm, a strict single-spike constraint is applied. A boolean mask tracks the firing history of every neuron; once a neuron exceeds its threshold, it emits a spike, records its timestamp, and is permanently locked out from firing again for the remainder of the 64-tick saccade. This eliminates burst firing, guaranteeing maximum sparsity and preventing bus congestion. The final classification is determined by the output neuron that records the earliest spike timestamp.

5.3 • INFORMATION REPRESENTATION

The choice of neural code lays the foundation for information flow and dictates the efficiency of the entire system. While Rate Coding (encoding pixel intensity as spike frequency) is straightforward and simple to implement with standard Integrate-and-Fire neurons, it is inefficient compared to TTFS. Rate codes require integration over extended time windows to calculate an average, introducing latency and saturating the network bus with redundant spikes. Furthermore, on digital hardware rate coding imposes additional stress on the system due to rapid switching which is very bad for transistor power draw and bus congestion.

To maximize energy efficiency and processing speed, this implementation utilizes a TTFS temporal encoding. In this regime, a single spike carries the information payload. A high-intensity (bright) pixel triggers an early spike, while a low-intensity (dark) pixel triggers a late spike. This compresses the spatial information into a highly sparse, priority-driven queue; downstream neurons begin processing as soon as the most salient features arrive, without waiting for an entire frame to integrate.

As noted in Section 3.4, temporal codes suffer from Phase Ambiguity—downstream neurons need a reference "clock" to decode latency. To resolve this without relying on a rigid, global system clock, we simulate the biological concept of a saccade (the rapid movement of the eye to fixate on a target). The initial presentation of the image acts as a synchronized global event (t_0). All subsequent input spikes are evaluated relative to this saccade onset, providing a natural, biologically plausible temporal reference frame.

5.3.1 • ENCODING

Following the TTFS principles described in Section 3.4.2, we convert the continuous pixel intensities of the MNIST dataset into discrete spike trains. For a given input image, we extract the luminance of each pixel and normalize it to a bounded range, where $p_i \in [0, 1]$ (1 representing maximum intensity and 0 representing the background).

We implement two distinct conversion mappings to evaluate latency dynamics: Linear and Logarithmic. Let T_{max} represent the maximum allowed simulation window for a single inference step.

For the Linear mapping, the spike latency t_i is inversely proportional to the pixel intensity:

$$t_i = T_{\text{max}} - (T_{\text{max}} \cdot p_i)$$

Equation 9: Linear Intensity-to-Delay Encoding

For the Logarithmic mapping, the delay is scaled logarithmically, which allocates higher temporal resolution to brighter pixels, further segregating the most salient features at the start of the simulation window:

$$t_i = T_{\max} - (T_{\max} \cdot (\log(1 + p_i))(\log(2)))$$

Equation 10: Logarithmic Intensity-to-Delay Encoding

Under both mappings, the brightest pixels fire immediately near $t = 0$, transmitting the most critical structural features of the digit first, while background pixels are suppressed.

5.3.2 • DECODING AND NEURON MODELS

Decoding the temporal information generated by the TTFS encoding requires a neuron model capable of accumulating discrete events. However, a fundamental engineering tension exists between biological fidelity, computational efficiency, and the ability to strictly decode temporal order.

To systematically evaluate these trade-offs, this thesis implements and benchmarks four distinct neuron models. These models range from simple arithmetic accumulators requiring global synchronization to complex, fully asynchronous dynamic systems. Each model processes incoming spikes by updating its membrane potential $V_{m(t)}$ when an event arrives at time t, carrying a synaptic weight w_i.

MODEL A: THE SIMPLE WINDOW INTEGRATOR (STANDARD IF)

The most computationally lightweight approach is the standard Integrate-and-Fire (IF) model without any leak or decay mechanisms. In this paradigm, the neuron acts as a pure arithmetic accumulator during the simulation window.

$$V_{m(t)} = V_{m(t_{\text{prev}})} + w_i$$

Equation 11: Simple Window Integration

Computational Cost: This model is extremely cheap to execute on digital hardware, requiring only a single addition operation $O(1)$ per incoming spike. **Temporal Decoding Capability:** While highly efficient, this model is theoretically flawed for pure TTFS decoding. It cannot differentiate the relative order of incoming signals. If Spike A ($w = 5$) arrives at $t = 1$ and Spike B ($w = 2$) arrives at $t = 10$, the final potential is identical to the reverse arrival order. **Synchronization:** Because the potential never naturally decays, this model relies entirely on a rigid, globally synchronized "saccade" (a hard reset of V_m to 0 at the end of the time window) to prevent the network from firing continuously due to lingering historical noise.

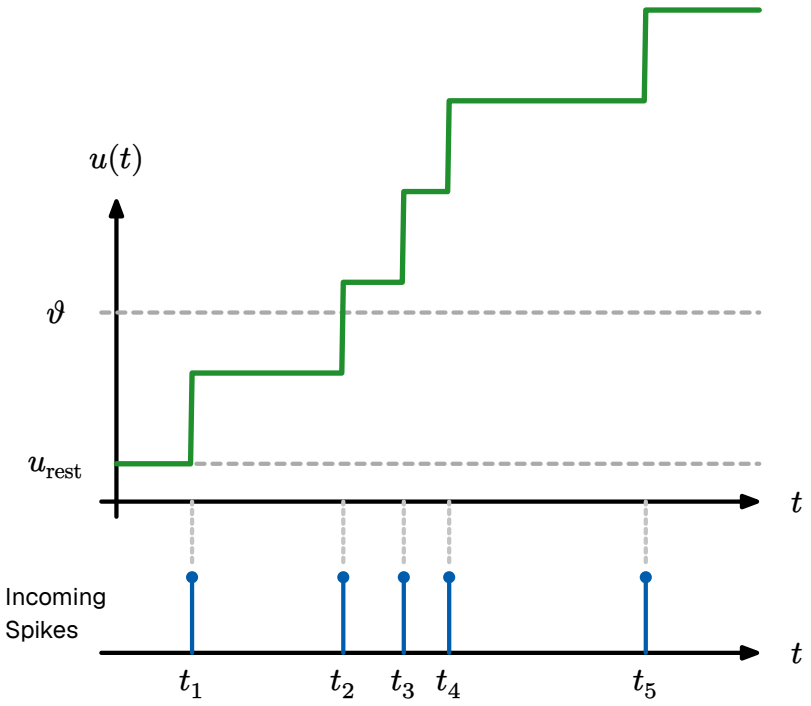

Figure 27: Network architecture. The 28x28 images are flattened and passed through a Fully Connected Network. Both the Artificial Neural Network (ANN) and Spiking Neural Network (SNN) share this identical macroscopic topology.

```
evaluate_model_A_IF(incoming_spikes, weights, threshold) → integer:
    sort(incoming_spikes, by_time)
    v_m = 0.0
    firing_time = None
    for spike in incoming_spikes:
        weight = weights[spike.source]
        v_m = v_m + weight
        if v_m ⩾ threshold:
            firing_time = spike.time
            break
    return firing_time
```

Algorithm 3: Model A: Simple Window Integrator (Standard IF)

MODEL B: THE STANDARD LEAKY INTEGRATE-AND-FIRE (LIF)

Model B—the LIF model is covered in great detail in Section 3.3.1, Model B fires only if the incoming spiketrain has a sufficient density of spikes. A sparse spiketrain can not overcome the expontially decaying membrane potential.

Computational Cost: This model is significantly more intensive, requiring the calculation of exponential functions for every discrete event, which consumes substantial clock cycles on standard arithmetic logic units.

Temporal Decoding Capability: The exponential leak naturally favors spikes that arrive in rapid succession, providing a basic temporal filter. However, standard LIF struggles to

strictly prioritize **order** in a TTFS scheme unless the time constant τ_m is perfectly tuned to the specific temporal distribution of the dataset.

Synchronization: Similar to Model A, while the leak reduces residual noise, it generally still requires a global saccade reset between distinct inference phases to guarantee a clean slate for the next image.

```
evaluate_model_B_LIF(incoming_spikes, weights, threshold, tau_m) →
integer:
    sort(incoming_spikes, by_time)
    v_m = 0.0
    t_prev = 0.0
    firing_time = None
    for spike in incoming_spikes:
        weight = weights[spike.source]
        delta_t = spike.time - t_prev
        v_m = max(0.0, v_m * exp(-delta_t / tau_m) + weight)
        if v_m ⩾ threshold:
            firing_time = spike.time
            break
        t_prev = spike.time
    return firing_time
```

Algorithm 4: Model B: Standard Leaky Integrate-and-Fire (LIF)

MODEL C: THE CURRENT-ACCUMULATING LINEAR RAMP

To enforce temporal order differentiation without the transcendental computational overhead of exponential kernels, Model C utilizes a linear time-dependent accumulator. In a TTFS encoding scheme, the relative latency of an afferent spike must dictate its magnitude of influence on the post-synaptic potential. This is achieved by modeling the membrane potential $u(t)$ as a system of coupled differential equations driven by a sequence of Dirac delta impulses:

$$\dot{I}(t) = \sum_i w_i \delta(t - t_i)$$

$$\dot{u}(t) = I(t) + \sum_i w_i \delta(t - t_i)$$

This formulation ensures that an afferent spike at time t_i with weight w_i exerts a dual influence: it induces an immediate translocation of the potential state while simultaneously incrementing the rate of change for all subsequent integration intervals. Because early spikes establish the baseline slope $I(t)$ for the remainder of the simulation window, they exert a disproportionate influence on the time-to-threshold. Consequently, the network becomes inherently sensitive to rank-order inputs.

$$I(t_i^+) = I(t_{i-1}^+) + w_i$$

$$u(t_i^+) = u(t_{i-1}^+) + I(t_{i-1}^+) \cdot (t_i - t_{i-1}) + w_i$$

Equation 12: Discrete Recurrence Relations for Model C

Computational Complexity: Optimized. The model replaces power-intensive transcendental operations with floating-point additions and a single multiplication per spike event ($I \cdot \Delta t$).

Temporal Decoding: High. The quadratic-like growth of the potential relative to spike arrival times allows for precise differentiation of closely timed input sequences.

Synchronization: This model requires a rigorous global synchronization protocol (e.g., a saccade-driven clock). In the absence of a periodic global reset to zero the state variables $I(t)$ and $u(t)$, the linear integration would diverge toward hardware saturation limits.

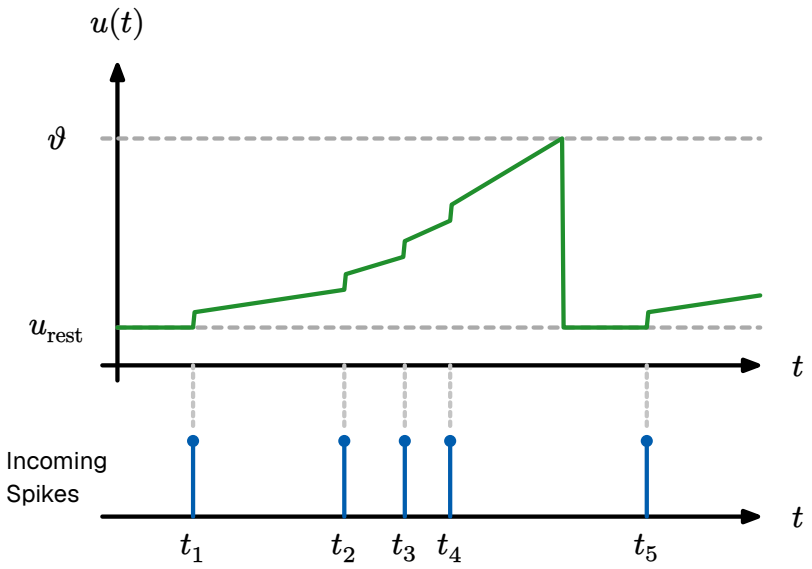

Figure 28: Evolution of state variables in Model C. This demonstrates the resulting piecewise linear membrane potential $u(t)$. Note how earlier spikes increase the integration gradient, accelerating the trajectory toward the firing threshold θ.

```
linearRampNeuron(S, W, theta) → t_fire:
    u ← 0.0
    I ← 0.0
    t_prev ← 0.0
    for each (t_i, w_i) ∈ S:
        Δt ← t_i − t_prev
        u ← u + (I · Δt) + w_i
        I ← I + w_i
        if u ≥ θ:
            return t_i
        t_prev ← t_i
    return ∞
```

Algorithm 5: Model C: Discrete State Update Algorithm

MODEL D: THRESHOLD-SENSITIVE INTEGRATION (STATE-DEPENDENT DISCOUNTING)

Drawing inspiration from the adaptation mechanisms of the GLIF model (Section 3.3.2), Model D introduces a state-dependent penalty to incoming spikes. Rather than penalizing spikes based on the passage of time, this model penalizes spikes based on the current internal state of the neuron.

In this paradigm, the increase in membrane potential is inversely proportional to the current potential itself. When a spike arrives at a neuron in its resting state ($V_m = 0$), there is no discount, and the full synaptic weight is added. However, if a spike arrives when the neuron is already close to the firing threshold, its impact is exponentially discounted.

$$V_{m(t)} = V_{m(t_{\text{prev}})} + w_i \cdot \exp\Big(-\gamma\Big(V_{m(t_{\text{prev}})}\Big)(V_{\text{th}})\Big)$$

Equation 13: State-Dependent Weight Discounting

From a computational perspective, this approach imposes a moderate cost. While it requires an exponential calculation (or a linear approximation), it does not need to continuously track the elapsed time (Δt) between individual spikes, saving memory overhead compared to continuous-time dynamic models. Furthermore, this mechanism excels at temporal decoding because it inherently prioritizes the sequence of arrival. The earliest spikes encounter an "empty" neuron and contribute their absolute maximum weight. Spikes arriving later encounter a partially filled potential and are severely dampened. Consequently, high-weight spikes must arrive early to have a meaningful impact, naturally aligning with the TTFS encoding hierarchy. Because this model drops the continuous temporal leak in favor of event-driven weight scaling, the potential does not naturally decay to zero between images. Therefore, much like Models A and C, it relies on a global "saccade" reset at the conclusion of the simulation window to clear the internal state for the next inference phase.

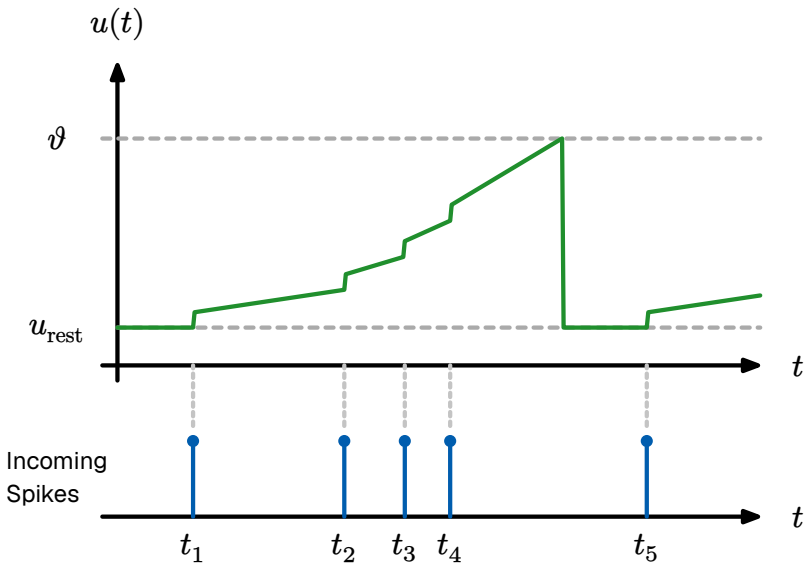

Figure 29: Neuron model where new incoming spikes have an exponentially decaying influence based on the current potential.

```
evaluate_model_D_Discount(incoming_spikes, weights, threshold, gamma)
→ integer:
    sort(incoming_spikes, by_time)
    v_m = 0.0
    firing_time = None
    for spike in incoming_spikes:
        weight = weights[spike.source]
        discount_factor = exp(-gamma * (v_m / threshold))
        v_m = v_m + (weight * discount_factor)
        if v_m ⩾ threshold:
            firing_time = spike.time
            break
    return firing_time
```

Algorithm 6: Model D: Threshold-Sensitive Integration

5.3.3 • THRESHOLDING AND LATERAL INHIBITION

Regardless of the chosen internal integration dynamics, all four models share identical discrete output behavior. To implement the WTA dynamics established in Section 3.5.4, the output classification layer utilizes lateral inhibition. Following the integration step, the neuron evaluates its firing condition: if $V_{m(t)} > V_{\text{th}}$, an output spike is emitted, and the internal state variables (V_m, and I if applicable) are reset. Furthermore, if the neuron belongs to the output classification layer, its spike triggers a WTA condition, instantly suppressing all competitors to finalize the classification.

5.4 • TRAINING METHODOLOGIES

Following the optimization principles established in Section 4.1, our training methodology combines weight-based synaptic plasticity with topological structural plasticity. In a neuromorphic context, learning is not merely parameter tuning but a physical self-organization of the network. We implement a dual-stage learning process: unsupervised feature discovery via local plasticity rules, followed by a global structural optimization phase.

5.4.1 • STRUCTURAL PLASTICITY AND SYNAPTOGENESIS

Drawing from the biological concept of synaptogenesis (Section 3.6.1), our model begins with a sparse set of potential connections. During the initial training phase, pre-synaptic activity triggers the probabilistic creation of new synapses. This "random growth" mechanism is computationally efficient and highly parallelizable on digital hardware.

To ensure functional efficiency, we implement a "synaptic gravity" heuristic. If a post-synaptic neuron already exhibits strong activation, it exerts a higher probability of attracting new connections from coincident pre-synaptic sources. This effectively reinforces emerging patterns, allowing the network to converge on salient features more rapidly than a purely random growth model.

5.4.2 • PATTERN PREDICTION AND ORDER DECODING

The core of our learning objective is the detection of spatiotemporal sequences. In a Time-to-First-Spike (TTFS) paradigm, the relative arrival order of spikes defines the pattern identity. We hypothesize that a neuron successfully learns a pattern (e.g., sequence $A \to B \to C$) when it adjusts its internal thresholds to fire as soon as the first sufficient evidence ($A \to B$) arrives.

However, this early firing introduces a prediction risk: if the neuron fires based on a prefix ($A \to B$) but the expected suffix (C) fails to arrive, the synaptic efficacy must be penalized. This mimics the biological concept of error-driven learning without a global supervisor; the neuron "predicts" the completion of a learned pattern and self-corrects based on subsequent local evidence. If the predicted input is absent, the weights associated with the prefix are slightly decayed, preventing the network from becoming overly sensitive to incomplete or noisy patterns.

5.4.3 • COMPETITIVE SPECIALIZATION

To prevent all neurons from converging on the same dominant features, we utilize the lateral inhibition motifs described in Section 3.5.4. When a neuron successfully classifies a pattern and fires, its inhibitory output suppresses neighboring units. This forces competing neurons to specialize in distinct, non-overlapping geometric

primitives. In our implementation, this competition is governed by the relative timing of spikes; the fastest neuron to recognize a feature "wins" the representation, while others are forced to adapt to secondary or tertiary patterns in the data stream.

5.4.4 • WEIGHT QUANTIZATION AND STABILITY

To bridge the gap between continuous simulation and physical neuromorphic hardware, we evaluate learning across varying degrees of weight resolution. While standard STDP assumes continuous weight updates, we implement a quantized learning scheme where synapses are restricted to binary or low-bit integer states. This mimics the constraints of memristive crossbar arrays, where weights are represented by discrete conductance levels. Stability is maintained through homeostatic scaling (Section 3.6.5), ensuring that the total synaptic drive to any individual neuron remains within a fixed dynamic range regardless of the connection density.

5.4.5 • WEIGHT TRANSFER

Following the theory of offline training and mapping discussed in Section 4.5.1, direct one-to-one transfer of floating-point weights from an ANN to an SNN often results in catastrophic failure; the continuous activation scales do not naturally align with discrete spiking thresholds. Furthermore, physical neuromorphic hardware (such as crossbar arrays) imposes strict limitations on synaptic weight resolution, rarely supporting 32-bit floating-point numbers.

To emulate these hardware constraints and ensure threshold compatibility, the continuous weights of the trained ANN (W_{ANN}) undergo a pseudo-INT8 quantization process before being loaded into the SNN simulator. The weights are multiplied by a static scaling factor of 64, rounded to the nearest integer, and clamped to an 8-bit signed integer range $[-128, 127]$:

$$W_{\text{SNN}}^{\{(l)\}} = \max\big(-128, \min\big(127, \text{round}\big(W_{\text{ANN}}^{l} \cdot 64\big)\big)\big)$$

Equation 14: Weight Scaling and Quantization

This transformation maps a standard continuous weight of 2.0 to the maximum synaptic efficacy of 128. By porting these quantized discrete weights (W_1 and W_2) directly to the GPU, the simulator strictly enforces the memory boundaries of physical neuromorphic silicon.

```
start with a collection of neurons with arbitrary connections
if a pre-synaptic neuron fires then
    it has a chance to grow a synapse to a random post-synaptic neuron
if a post-synaptic neuron fires then
    strengthen all connections to pre-synaptic neruons that fired before
    remove connections to pre-synaptic neurons that did not fire or fired after
    a neuron can be both pre-synaptic and post-synaptic
```

Algorithm 7: Unsupervised local learning rule for induvidual neurons. Based on Spike-Timing-Dependent Plasticity (STDP)

```
probability of growing a synapse is inversely
proportional to the amount it already has
earlier firings should get a better chance to grow synapses,
although this is regulated by inhibitory action
```

Algorithm 8: Growing rules for synapses

5.4.6 • TTFS STDP INSPIRED LEARNING RULE

Our learning rule adapts the STDP mechanism from Section 3.6.4 to the TTFS domain. In standard continuous networks, synaptic weights are updated via global error gradients. In the STDP paradigm, a synapse w_{ij} connecting a pre-synaptic neuron i to a post-synaptic neuron j is updated based strictly on the temporal difference between their respective firing times.

Let t_i denote the spike time of the input neuron, and t_j denote the spike time of the output neuron. The relative arrival time is defined as $\Delta t = t_j - t_i$. Because this architecture utilizes a strict TTFS encoding where neurons fire at most once per saccade, the classical continuous STDP curve is adapted into a discrete, deterministic update rule:

- **Long-Term Potentiation (LTP):** If $t_i < t_j$, the pre-synaptic spike arrived before (or exactly at) the moment the post-synaptic neuron fired. This indicates causality. The synapse is strengthened, with the magnitude of the update decaying exponentially the further apart the spikes occurred.
- **Long-Term Depression (LTD):** If $t_i > t_j$, the pre-synaptic spike arrived after the post-synaptic neuron had already fired. The input was irrelevant to the decision, and the synapse is subsequently weakened.
- **Unused Synapses (Penalty):** If a pre-synaptic neuron fires but the post-synaptic neuron never fires during the saccade, a slight negative decay is applied to encourage forgetting of dead connections.

$$\Delta w_{\{ij\}} = \begin{cases} A_+ \cdot \exp\left(-\left(t_j - t_i\right)/\tau_+\right) & \text{if } \ t_i < t_j \ \text{(LTP)} \\ -A_- \cdot \exp\left(-\left(t_i - t_j\right)/\tau_-\right) & \text{if } \ t_i > t_j \ \text{(LTD)} \end{cases}$$

Equation 15: Additive STDP Weight Update

To prevent runaway synaptic growth or catastrophic sign-flipping, the updated weights are strictly clamped to a positive physical range $w_{ij} \in [0, W_{\max}]$. Algorithm 2 details the exact logical implementation of this rule.

```
apply_stdp(t_pre, t_post, current_weight, A_plus, A_minus, tau, W_max):
    if t_post == -1:
        return max(0.0, current_weight - (A_minus * 0.1))
    delta_t = t_post - t_pre
    if delta_t >= 0:
        update = A_plus * exp(-delta_t / tau)
        new_weight = current_weight + update
    else:
        update = A_minus * exp(delta_t / tau)
        new_weight = current_weight - update
    return clamp(new_weight, min=0.0, max=W_max)
```

Algorithm 9: The TTFS STDP Weight Update Function

5.4.7 • TRAINING LOOP

To execute unsupervised feature extraction on the MNIST dataset, the STDP rule is embedded within the saccade simulation loop. The network is initialized with randomized synaptic weights $W \sim \mathcal{U}(0, 1)$.

During the training phase, an image is presented, and the network executes a forward pass. However, unlike the zero-shot inference phase, lateral inhibition (Winner-Takes-All) plays a critical structural role during training. When an output neuron fires, it aggressively inhibits its neighbors. This competitive dynamic forces different neurons to specialize in different geometric features; if one neuron learns to recognize a “loop,” the WTA mechanism prevents other neurons from learning that exact same redundant feature.

At the conclusion of the 64-tick saccade, the simulator halts, compares the timestamp arrays of the hidden and output layers, and computes the STDP weight updates in parallel before the next image is presented. Algorithm 3 outlines this autonomous learning pipeline.

```
  train_unsupervised_epoch(dataset, W1, W2, T_max):
1   for image in dataset:
2     spikes_pre = encode_to_ttfs(image)
3     spikes_post = simulate_saccade_wta(spikes_pre, W1, W2, T_max)
4     for i in range(num_input_neurons):
5       for j in range(num_output_neurons):
6         t_pre = spikes_pre[i]
7         t_post = spikes_post[j]
8         W1[i][j] = apply_stdp(t_pre, t_post, W1[i][j], …)
9     W1 = enforce_homeostatic_normalization(W1)
10  return W1
```

Algorithm 10: Unsupervised Spiking Neural Network (SNN) Training Pipeline

5.5 • EVALUATION METRICS

To rigorously validate the proposed Spiking Neural Network architectures, the evaluation framework must measure two distinct domains: **Classification Effectiveness** (accuracy and temporal decoding) and **Computational Efficiency** (resource usage and sparsity). Because the networks are evaluated across both supervised transfer and unsupervised native learning phases, specialized metrics are required for each.

5.5.1 • EFFECTIVENESS AND CLASSIFICATION PERFORMANCE

For Phase II (Zero-Shot Weight Transfer), evaluating classification performance is straightforward. The SNN is tasked with classifying the 10,000-image MNIST test set, and performance is measured using standard statistical metrics:

- **Top-1 Accuracy:** The percentage of images where the first output neuron to fire (via the Winner-Takes-All mechanism) corresponds to the correct ground-truth label.
- **Confusion Matrices:** Utilized to identify specific class overlaps (e.g., distinguishing a '4' from a '9'). Because the SNN utilizes discrete temporal thresholds rather than continuous probabilities, confusion matrices help identify if certain geometric features are disproportionately lost during INT8 quantization.

However, evaluating Phase III (Unsupervised STDP) requires a different approach. Because the network learns without labels, the output neurons do not inherently map to digits 0 through 9; they map to arbitrary geometric clusters. To evaluate classification accuracy on an unsupervised model, we employ a **Post-Hoc Labeling** strategy:

1. **Freezing:** After the STDP training phase concludes, synaptic plasticity is disabled (learning rate set to zero).

2. **Assignment:** A subset of the labeled training data is passed through the network. The simulator tracks the firing distribution of each output neuron. Each output neuron is permanently assigned the label of the digit class that most frequently triggered it to fire.
3. **Testing:** The standard 10,000-image test set is then passed through the network, and Top-1 accuracy is calculated using these newly assigned post-hoc labels.

5.5.2 • COMPUTATIONAL EFFICIENCY (HARDWARE PROXIES)

While the primary goal of neuromorphic engineering is immense energy reduction, evaluating true physical power draw (Joules-per-inference) requires deploying the network on dedicated silicon (such as Intel Loihi or IBM TrueNorth). Because this thesis utilizes a PyTorch-based software simulator running on standard von Neumann hardware (GPUs), the simulator actually consumes **more** energy than a standard ANN due to the overhead of the temporal event loop.

To mathematically isolate and theorize the energy savings of the underlying **algorithm**, we abandon direct power measurements and utilize hardware-agnostic proxy metrics universally recognized in SNN literature:

5.5.3 • SPARSITY AND SYNAPTIC OPERATIONS (SYOPS)

In a standard Artificial Neural Network, every forward pass forces every neuron to compute a dense Multiply-Accumulate (MAC) operation, regardless of the input data. The computational cost is fixed.

In a Spiking Neural Network, computation only occurs when a spike is emitted. By tracking the total number of spikes generated across the hidden layer (N_{spikes}) during a single 64-tick saccade, we can calculate the network's spatial and temporal sparsity.

Because Integrate-and-Fire neurons do not require multiplication (a spike simply adds its weight to the potential), MAC operations are replaced by simpler Synaptic Operations (SyOPs), which are purely arithmetic additions. The computational cost of a single inference step in the SNN is estimated as:

$$\text{Total SyOPs} = \sum_{\{l=1\}}^{\{L\}} N_{\text{spikes}}^{\{(l)\}} \cdot F_{\text{out}}^{\{(l)\}}$$

Equation 16: Spiking Neural Network (SNN) Operational Cost Proxy

Where F_{out} is the fan-out (number of outgoing synaptic connections) of the spiking neurons. By comparing the total SyOPs of the SNN against the fixed MAC count of the baseline ANN, we can derive a theoretical energy efficiency ratio. If the TTFS encoding is highly sparse, the SyOP count should be orders of magnitude lower than the ANN MAC count.

5.5.4 • TEMPORAL LATENCY METRICS

Finally, to evaluate the specific efficacy of the TTFS encoding and the four distinct neuron models, we measure **Time-to-Decision Latency**.

Latency is defined as the exact simulation tick $t \in [0, 64)$ at which the WTA output neuron fires. A lower average latency indicates a highly efficient temporal decoder that successfully prioritizes salient information, allowing the system to power down or reset earlier in the simulation window. Conversely, if a neuron model consistently requires the full 64 ticks to reach a decision, it fails to capitalize on the advantages of the temporal priority queue.

5.5.5 • QUALITATIVE EVALUATION VIA LATENT SPACE PROJECTION (T-SNE)

While post-hoc labeling provides a quantitative measure of classification accuracy, it inherently forces discrete semantic labels onto continuous geometric representations. To truly understand the internal representations learned by the unsupervised network, we must evaluate the topology of the hidden layer's latent space.

In the proposed architecture, the hidden layer consists of $N_1 = 128$ neurons. Each MNIST image produces a unique 128-dimensional spike-time signature. To qualitatively visualize how the network groups these high-dimensional signatures, we employ t-distributed Stochastic Neighbor Embedding (t-SNE). This non-linear dimensionality reduction technique projects the 128-dimensional manifold down to a 2D plane, preserving local proximities such that similar activation patterns appear as localized clusters.

This visualization provides a critical comparative tool between the supervised and unsupervised models:

- **Supervised Latent Space (Phase II):** In the weight-transferred network, the continuous weights were explicitly optimized via cross-entropy loss to separate the 10 categorical digit classes. Consequently, the t-SNE projection is expected to reveal highly segregated clusters corresponding strictly to semantic labels (digits 0 through 9).
- **STDP Latent Space (Phase III):** In contrast, the native STDP algorithm possesses no semantic knowledge of the dataset. It operates purely as a temporal coincidence detector, strengthening synapses when overlapping geometric features (such as lines or curves) trigger coincident pre-synaptic spikes.

Therefore, the STDP t-SNE projection is hypothesized to cluster images based on morphological similarity rather than strict categorical class. Digits that share fundamental structural primitives (e.g., the top-loops of '8's and '9's, or the straight vertical edges of '1's and '7's) should form contiguous manifolds in the latent space. Demonstrating this geometric clustering via t-SNE will empirically validate that the

local STDP rule successfully self-organized the network into a biologically plausible feature extractor.

5.6 • EXPERIMENT SETUP AND EVALUATION PHASES

To systematically evaluate the theoretical claims, computational trade-offs, and classification performance of the proposed Spiking Neural Network architectures, the experiment is structured into three distinct execution phases. This tri-phasic approach isolates the specific variables of temporal decoding, offline parameter transfer, and native online learning.

5.6.1 • PHASE I: SYNTHETIC TEMPORAL BENCHMARKS

Before evaluating the networks on high-dimensional visual data, it is necessary to empirically validate the temporal decoding capabilities of the four neuron models (Simple IF, LIF, Linear Ramp, and Asynchronous Decay) established in Section 3.3.2.

The objective of this phase is to test whether the models can successfully differentiate between input patterns that possess identical spatial weights but differ purely in their temporal order of arrival. The models are subjected to synthetic micro-tasks consisting of controlled spike trains:

- **Permutation Sensitivity Test:** A target neuron is presented with a sequence of spikes (e.g., Spike A at $t = 1$, Spike B at $t = 5$). The sequence is then reversed (Spike B at $t = 1$, Spike A at $t = 5$). The models are evaluated on their ability to reach the firing threshold for the correct sequence while remaining sub-threshold for the reversed sequence.
- **Coincidence Detection Test:** Spikes are injected with varying temporal spread (Inter-Spike Intervals). The models are evaluated on their ability to act as coincidence detectors, firing only when inputs arrive within a tight temporal window, demonstrating robustness against background noise.

This phase serves as a functional unit test, mathematically verifying which models are viable for strict TTFS decoding before they are deployed on the MNIST dataset.

5.6.2 • PHASE II: ZERO-SHOT INFERENCE VIA WEIGHT TRANSFER

The second phase evaluates the performance of the spiking models on real-world visual data using the offline ANN-to-SNN weight transfer methodology detailed in Section 3.4.

A baseline ANN with the identical $784 \to 128 \to 10$ Fully Connected topology is trained on the MNIST dataset using standard Backpropagation and Cross-Entropy Loss until convergence. The learned floating-point weights are then quantized, scaled to the INT8 range $[-128, 127]$, and loaded directly into the SNN.

The SNN is then tasked with classifying the unseen 10,000-image MNIST test set without any further learning. During this phase, the four neuron models are benchmarked against the baseline ANN on three critical metrics:

1. **Classification Accuracy:** The percentage of images correctly identified, measuring how much accuracy was lost during the quantization and temporal translation.
2. **Latency (Time-to-Decision):** The average number of simulation ticks required for the output layer to trigger the Winner-Takes-All mechanism, measuring the speed of the TTFS encoding.
3. **Sparsity (Energy Efficiency Proxy):** The total number of spikes generated across the hidden layer per inference. Models requiring fewer spikes to reach a decision demonstrate higher theoretical energy efficiency for neuromorphic hardware.

5.6.3 • PHASE III: NATIVE UNSUPERVISED LEARNING (STDP)

While Phase II evaluates the inference capabilities of the hardware, it relies on global, offline backpropagation. To achieve true neuromorphic autonomy, the network must be capable of adapting to stimuli locally and dynamically.

The final phase discards the pre-trained ANN weights entirely. The SNN is initialized with randomized synaptic weights and subjected to the MNIST dataset utilizing a biologically inspired, unsupervised learning rule based on STDP.

Because this phase is unsupervised, the network is not provided with the correct digit labels. Instead, the objective is to evaluate whether the local STDP rules, combined with lateral inhibition, can spontaneously self-organize the hidden layer neurons to cluster distinct geometric features (such as loops and edges) purely based on the temporal correlations of the input spikes. The performance of the four neuron models will be analyzed to determine which internal dynamics best support stable, unsupervised feature extraction.

6 • RESULTS

6.1 • NERUON MODELS

6.2 • MNIST WITH COPIED WEIGHTS

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

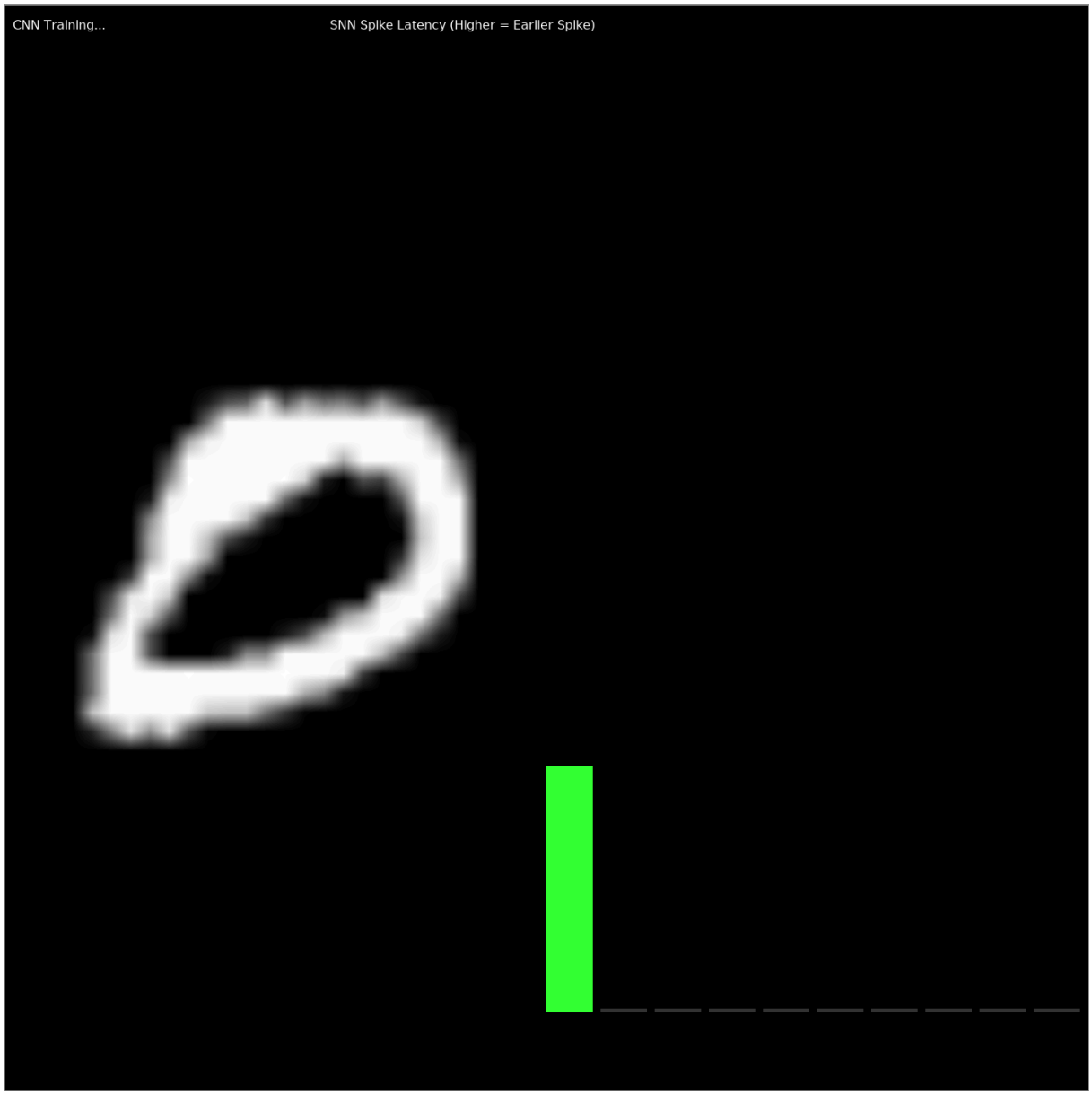

Figure 30: Neural network before learing

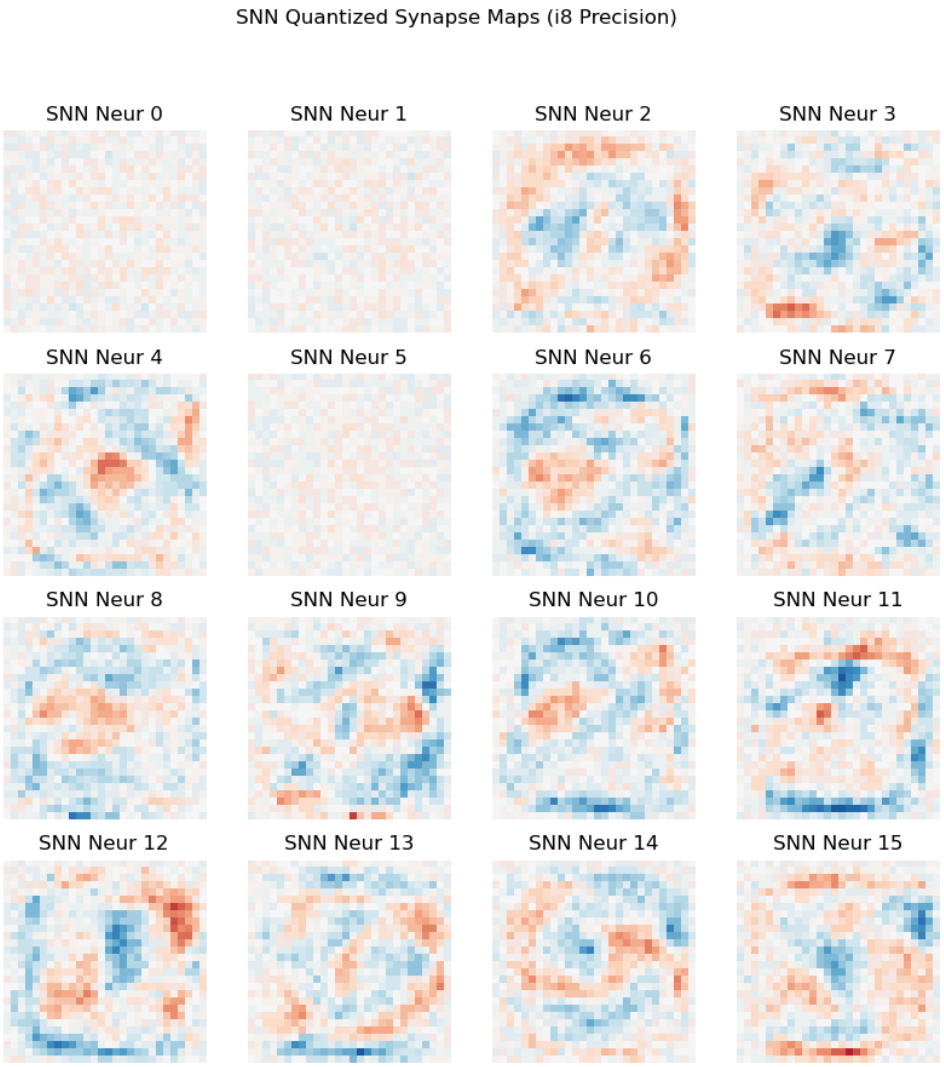

Figure 31: Neural network before learing

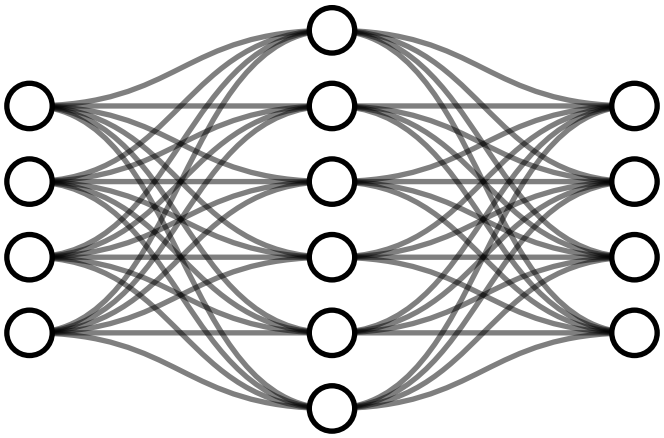

Figure 32: Neural network during learning

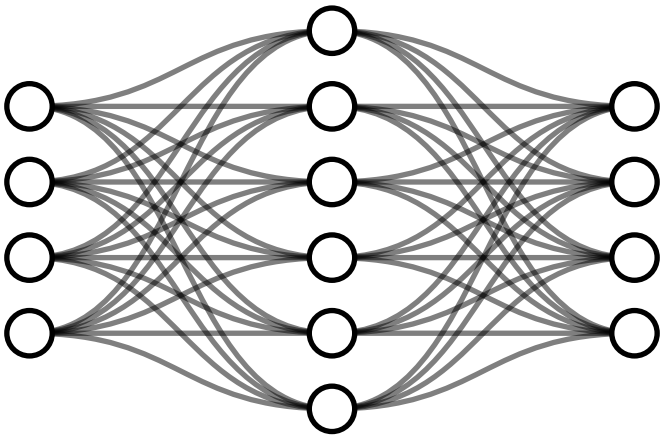

Figure 33: Neural network after learning

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

6.3 • MNIST WITH SNN TRAINED WEIGHTS

0	0	1	1
1	1	1	1

Table 1: Number of operations

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane

Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At

etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

7 • DISCUSSION

While the experimental results validate the core principles of TTFS encoding and spiking integration, extrapolating these methods from controlled benchmarks to real-world deployment reveals significant engineering bottlenecks. This chapter critically analyzes the limitations observed during the implementation phase, specifically regarding data encoding, spatial representation, architectural translation, and hardware constraints.

7.1 • ENCODING MODALITIES AND THE CONTRAST PROBLEM

In the engineered test environment, absolute pixel intensity was mapped directly to spike latency. For a highly controlled toy dataset like MNIST, this approach yields adequate performance, as the digits are isolated in high-contrast, low-resolution environments. However, absolute luminance is a notoriously brittle feature for real-world computer vision.

Robust biological and artificial vision systems rely on local contrast (the relative difference between adjacent pixels) to determine object boundaries, as contrast remains invariant under shifting global illumination. Attempting to compute true, normalized contrast directly within a TTFS spiking network presents a severe temporal bottleneck. To accurately assess relative darkness in a purely temporal code, the downstream neurons must wait for the slowest (darkest) signals to arrive, effectively nullifying the high-speed advantages of the TTFS priority queue.

Consequently, attempting to calculate contrast natively within the SNN layers is computationally wasteful. The optimal solution is to offload this processing to the sensory periphery. Dedicated neuromorphic sensors, such as Dynamic Vision Sensors (DVS), natively output logarithmic intensity differences. Because the difference in log-space mathematically corresponds to a true contrast ratio independent of absolute luminance, passing this pre-encoded contrast data directly into the TTFS network avoids the temporal delay problem entirely.

7.2 • REPRESENTING SPACE AND DIMENSIONALITY

Encoding precise spatial coordinates using a pure TTFS scheme proved inherently difficult. In biological visual and motor cortices, spatial locations are often represented via orthogonal population codes, where specific populations activate to indicate direction or position. However, these biological populations largely utilize

rate coding; the "intensity" or certainty of a spatial position translates naturally into a higher firing frequency.

Conversely, TTFS is highly optimized for rapid, categorical decision-making (e.g., triggering a Winner-Takes-All classification), but struggles to map continuous numerical or spatial values without complex phase-ambiguity resolution. A potential architectural workaround is the implementation of hierarchical "space cells." Rather than mapping a massive 32×32 grid to individual temporal delays, the space could be subdivided into localized grids (e.g., overlapping 8×8 populations). This reduces the dimensionality of the temporal encoding, though it introduces resolution artifacts at the boundaries of the receptive fields.

7.3 • ARCHITECTURAL TRANSLATION (ANN TO SNN)

The zero-shot inference phase highlighted the frictions of translating classical architectures to spiking substrates. While the Fully Connected topology provided a transparent baseline, mapping state-of-the-art Convolutional Neural Networks (CNNs) is decidedly not a one-to-one process.

Classical continuous networks heavily utilize negative weights, static biases, and mathematical operations like Max Pooling. Neuromorphic systems handle these concepts fundamentally differently. An SNN replaces mathematical pooling with dynamic lateral inhibition, and negative continuous weights must be modeled via discrete inhibitory spike trains. These architectural mismatches dictate that SNNs cannot simply act as "drop-in" replacements for deep learning models; they require network topologies natively designed for event-driven dynamics.

7.4 • PLASTICITY DYNAMICS AND FORGETTING

During the evaluation of unsupervised learning, a fundamental tension emerged between the network's learning velocity and its stability. Unsupervised, local learning rules like STDP require aggressive hyperparameter tuning. If the plasticity rate is too high, the network adapts quickly but suffers from rapid catastrophic forgetting—overwriting previously learned geometric features when presented with novel stimuli. Conversely, if the plasticity is too low, the network fails to converge on meaningful representations within an acceptable time frame. Designing homeostatic mechanisms to stabilize this plasticity remains a core challenge for native learning.

7.4.1 • THE SPARSITY PARADOX AND GPU SIMULATION OVERHEAD

A core theoretical advantage of the proposed TTFS network is its extreme spatial and temporal sparsity. By design, only a small fraction of neurons emit spikes, mathe-

matically reducing the dense Multiply-Accumulate (MAC) operations of a standard ANN to a minimal set of Synaptic Operations (SyOPs).

However, evaluating this sparse algorithm on conventional GPUs introduces a significant "Sparsity Paradox." Standard deep learning frameworks (such as PyTorch) and standard GPU architectures are heavily optimized for dense, contiguous matrix-matrix multiplications via SIMD (Single Instruction, Multiple Data) execution. In the SNN simulation loop, sparsity is enforced via boolean masking (multiplying inactive neuron outputs by zero). While this successfully zeroes out the potential, the GPU Arithmetic Logic Units (ALUs) typically still execute the underlying floating-point multiplication cycle.

Consequently, the hardware does not naturally skip the computation for quiescent neurons unless specialized, unstructured sparse tensor kernels are deployed. When combined with the overhead of maintaining the temporal loop (the "saccade" ticks), the SNN simulator ironically consumes more absolute clock cycles and physical power on a GPU than the continuous ANN baseline.

This paradox highlights that true energy efficiency cannot be realized via matrix-masking on von Neumann hardware. Realizing the calculated SyOP savings requires deployment on native event-driven Neuromorphic ASICs (Application-Specific Integrated Circuits). These chips discard the matrix-multiplication paradigm entirely, utilizing Address Event Representation (AER) to asynchronously route data packets only when a spike physically occurs, reducing idle power draw to near zero.

7.4.2 • MITIGATION VIA SYNAPTIC PRUNING

While dynamic activation sparsity struggles on GPUs, structural sparsity offers a viable software-level mitigation. Future iterations of this work could introduce aggressive Synaptic Pruning, particularly following the unsupervised STDP learning phase.

Because STDP naturally depresses irrelevant synapses toward zero, a thresholding function could permanently sever these connections, converting the dense weight matrices ($W^{\{(1)\}}$, $W^{\{(2)\}}$) into highly sparse structures. By utilizing block-sparse tensor formats (e.g., Compressed Sparse Row), both software simulators and specialized sparse-accelerator chips can mathematically bypass the zero-weights, yielding physical reductions in memory bandwidth and computation even before transitioning to pure neuromorphic silicon.

7.5 • THE PHYSICAL HARDWARE GAP

Ultimately, the theoretical energy efficiency of neuromorphic algorithms is bounded by physical hardware. The connection density of the biological mammalian cortex

vastly exceeds the routing capabilities of modern CMOS (Complementary Metal-Oxide-Semiconductor) fabrication.

While the software simulations in this thesis demonstrate the algorithmic viability of sparse processing, achieving true biological efficiency requires novel materials. Non-volatile memory technologies, such as memristors and spintronics, offer the ability to physically colocate extreme-density analog weights with logic gates, perfectly mirroring biological synapses. Until these exotic substrates become commercially viable, the near-term future of applied neuromorphic computing lies in massively parallel, asynchronous digital ASICs designed using standard CMOS, serving as a transitional bridge to fully analog systems.

7.6 • FUTURE WORK

The findings and limitations discussed in this thesis present several promising avenues for future research in neuromorphic engineering:

- **Event-Based Dataset Validation:** Transitioning the experimental framework from static images (MNIST) to native temporal datasets, such as Neuromorphic-MNIST (N-MNIST) or DVS Gesture datasets, to fully exploit the asynchronous dynamics of the evaluated neuron models.
- **Surrogate Gradient Integration:** While this work focused on direct weight transfer and native STDP, future implementations should evaluate the efficacy of Surrogate Gradient Descent, combining the optimization power of backpropagation with the inference efficiency of spiking dynamics.
- **Hardware Deployment:** Porting the verified TTFS algorithms and current-accumulating neuron models from GPU simulation onto dedicated neuromorphic silicon (e.g., Intel Loihi) to empirically measure true joule-per-inference energy consumption against classical von Neumann baselines.

8 • CONCLUSION

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque

doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

BIBLIOGRAPHY

[1] ?, “Placeholder,” 2025.